



南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

CS304 SOFTWARE ENGINEERING

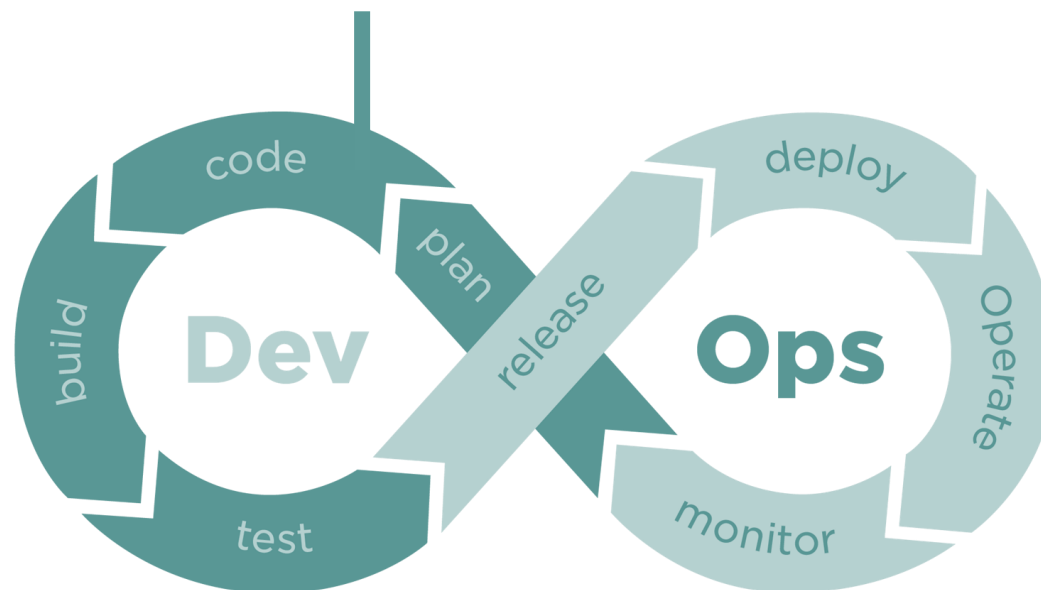
Yida Tao

taoyd@sustech.edu.cn



WHERE ARE WE NOW?

Version control systems



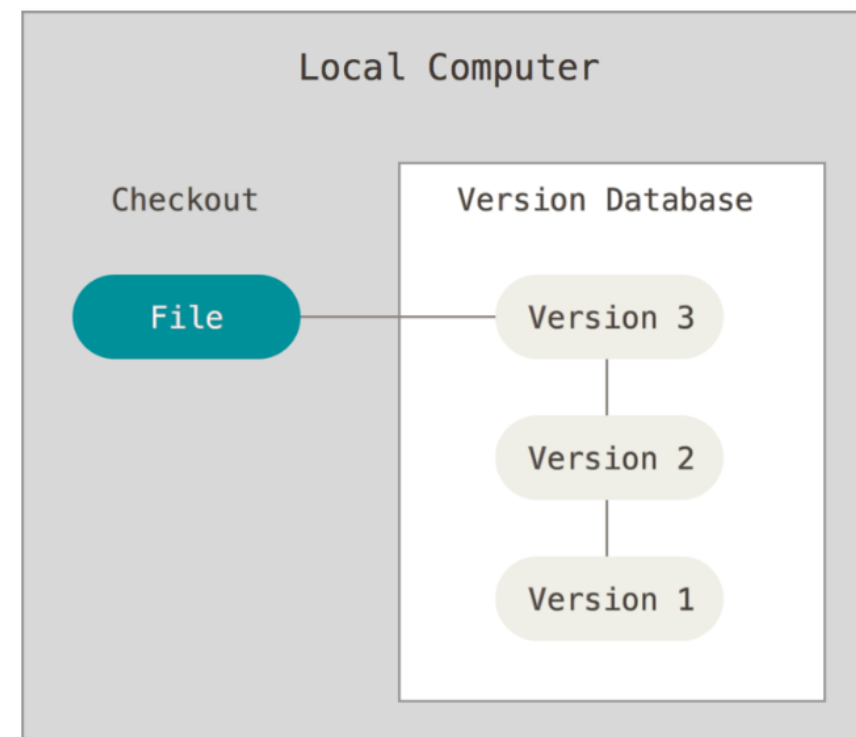


WHAT IS VERSION CONTROL?

- **Version control systems (VCS)** are software tools that help software teams track and manage modifications to artifacts (typically source code) over time.
 - **Revisit** older versions
 - **Revert** selected files back to a previous state and **recover** from mistakes
 - **Compare** changes over time,
 - **Diagnosing**: see **who** last modified something that might be causing a problem, who introduced an issue and **when**, and more.
- VCS are especially useful for DevOps teams since they help them to **reduce risks** caused by **continuous changes** and **increase successful deployments**.

LOCAL VERSION CONTROL SYSTEMS

- Local VCSs have a simple database that kept all the changes to files under revision control.
- Representative local VCS: RCS
- RCS works by keeping patch sets (i.e., the differences between files) in a special format on disk; it can then re-create what any file looked like at any point in time by adding up all the patches.

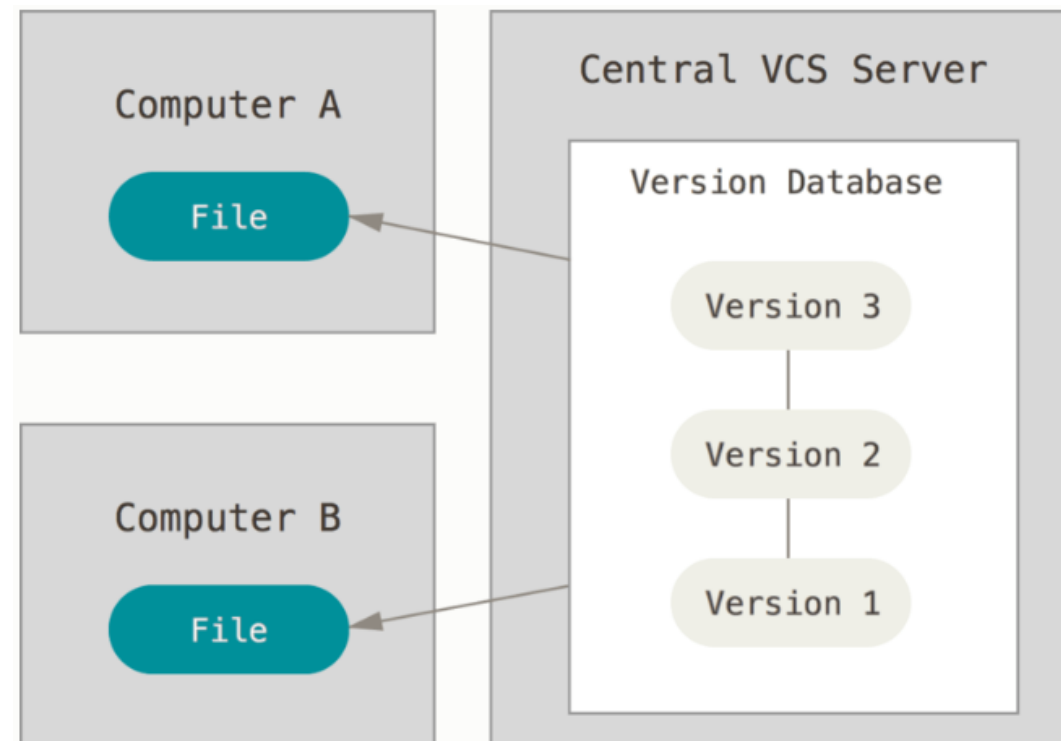


Problems?

<https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control>

CENTRALIZED VERSION CONTROL SYSTEMS

- Centralized VCS emerged since developers need to **collaborate** with each other
- Centralized VCS (e.g., CVS, Subversion/SVN) have **a single server** that contains all the versioned files; clients **check out** files from that central place.
- Using CVCS, everyone knows to a certain degree what everyone else on the project is doing.



<https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control>

Problems?



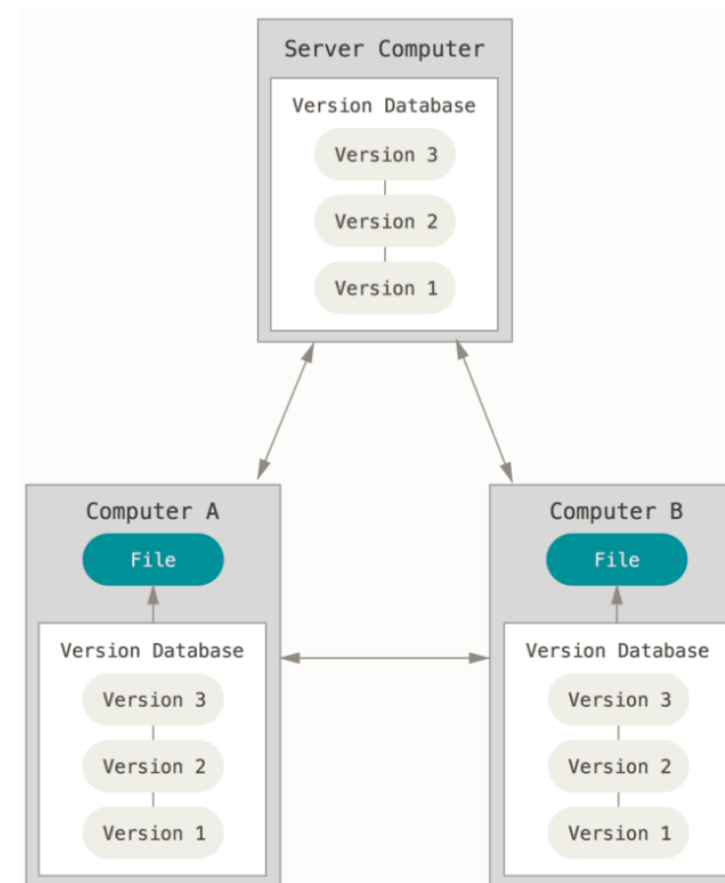
Linus Torvald (the creator of Linux & git) talk on centralized vcs & git

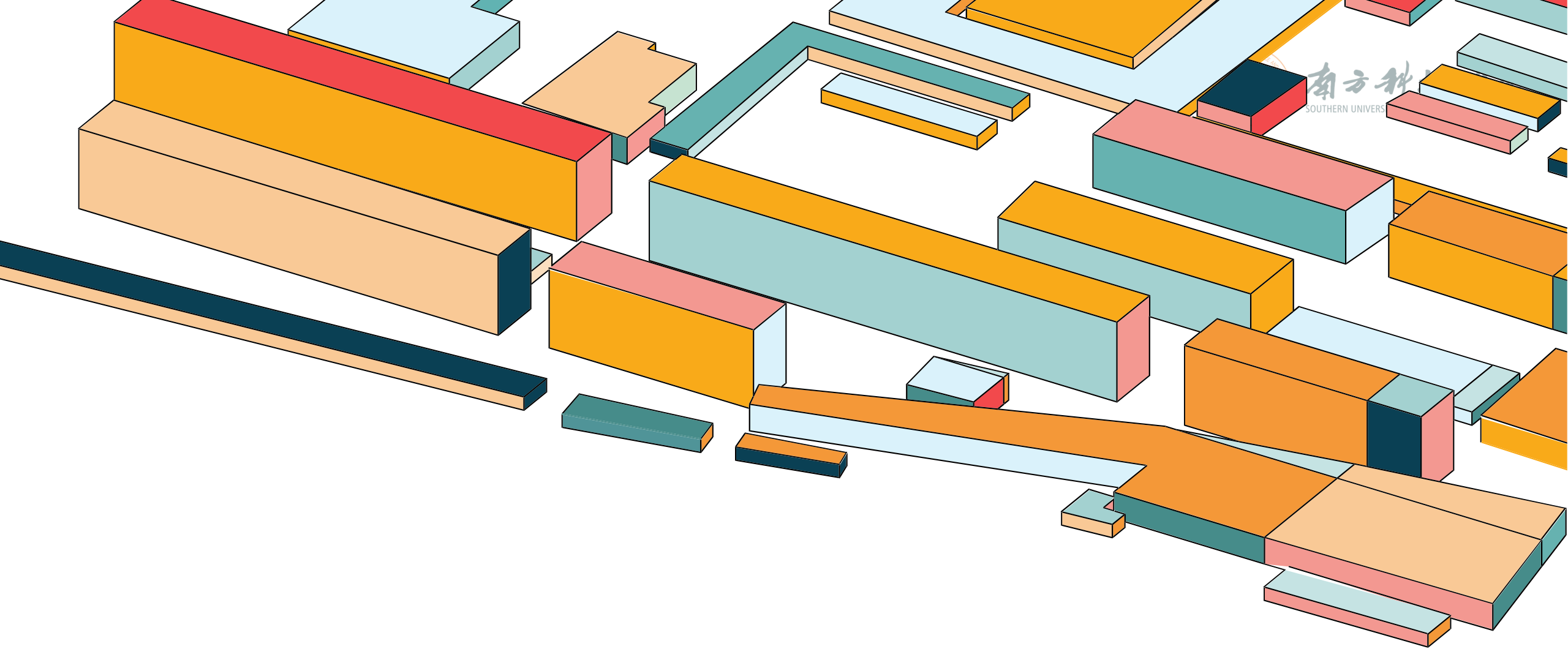
<https://www.youtube.com/watch?v=idLyobOhtO4>

DISTRIBUTED VERSION CONTROL SYSTEMS

- In a Distributed VCS (e.g., BitKeeper, git, Mercurial), clients don't just check out the latest snapshot of the files.
- Rather, they **fully mirror the repository**, including its **full history**. Every clone is really a full backup of all the data.
- Thus, if any server dies, and these systems were collaborating via that server, any of the client repositories can be copied back up to the server to restore it.

<https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control>





GIT ARCHITECTURE

GIT REPOSITORY



Local Repo

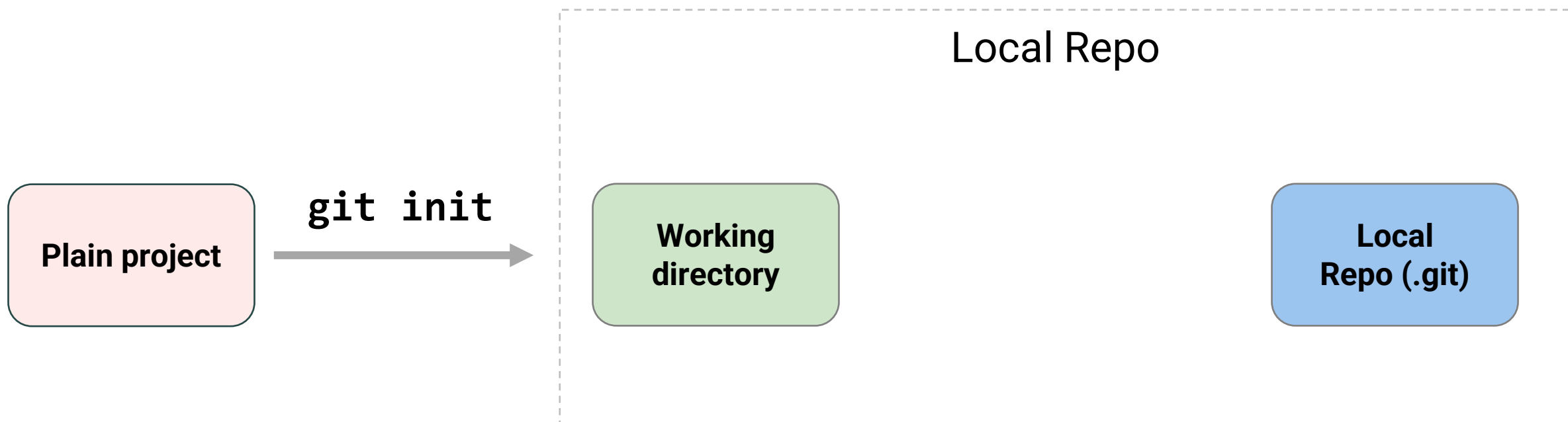
- The one on which we will make local changes, typically on our computer.
- Users have full git features on local git repos, even without Internet access

Remote Repo

- The one on a remote server (e.g., GitHub, GitLab, BitBucket)
- The purpose of a remote repository is to host/publish your code to the world and allow people to read or write it.



LOCAL REPO

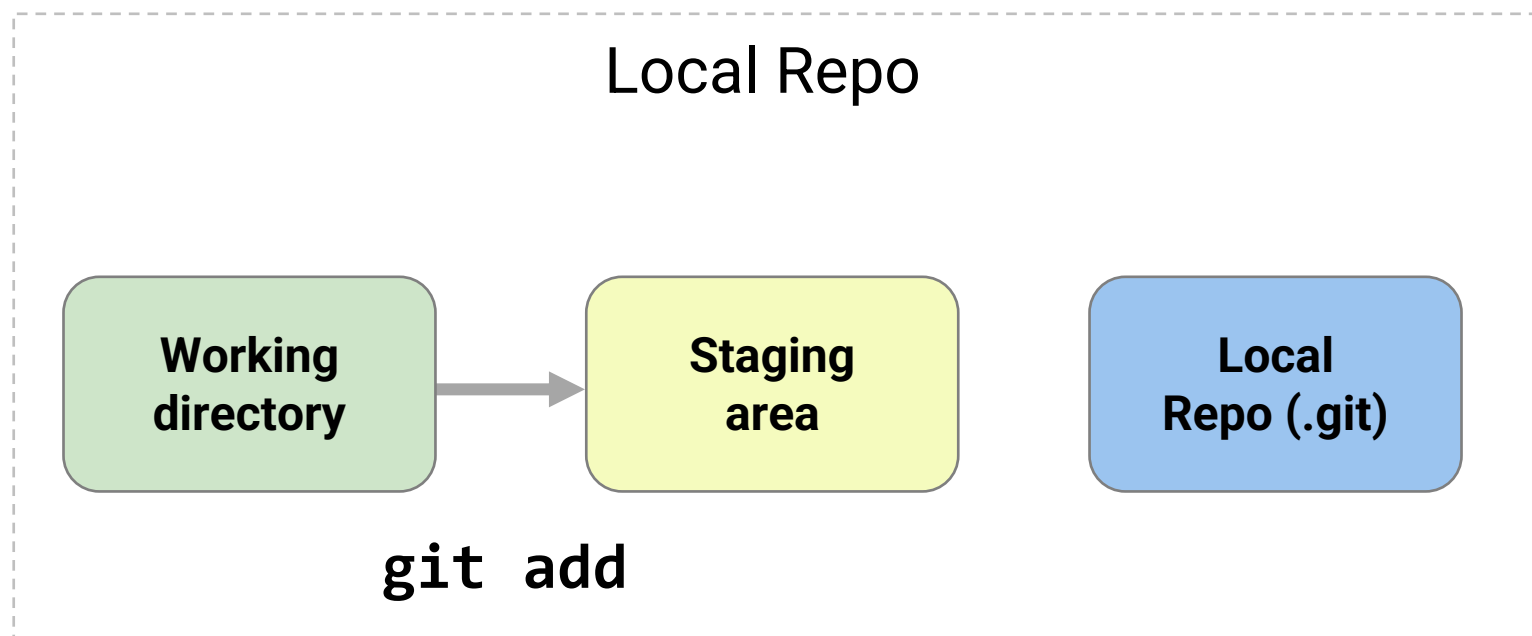




LOCAL REPO

- **Untracked** files are the ones still NOT tracked by Git. This is the state of new files you add to your repo.
- Git is aware the file exists, but still hasn't saved it in its internal database.
- If you lose the information from an untracked file, it's typically gone for good. Git can't recover it since it didn't store it in the first place.

To stage files to be part of the next commit, use `git add xxx`



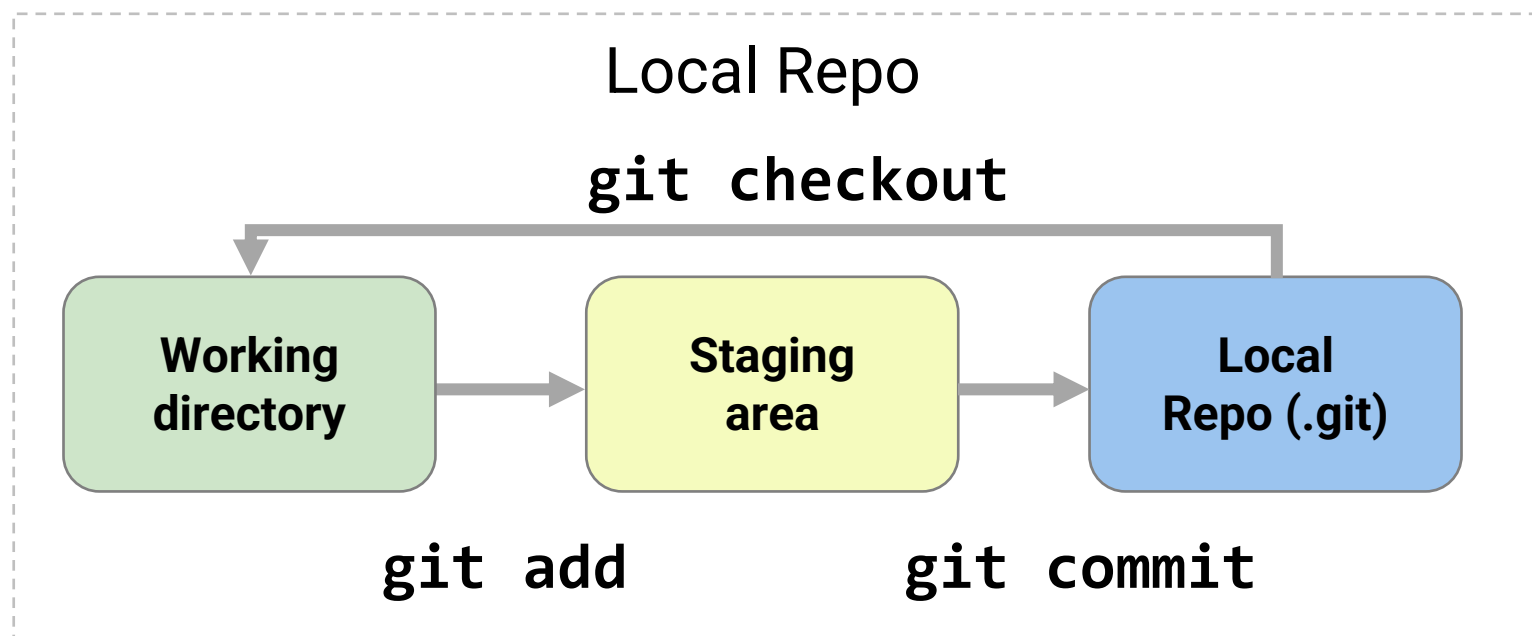


LOCAL REPO

Local repo: where Git stores the metadata and object database for your project.

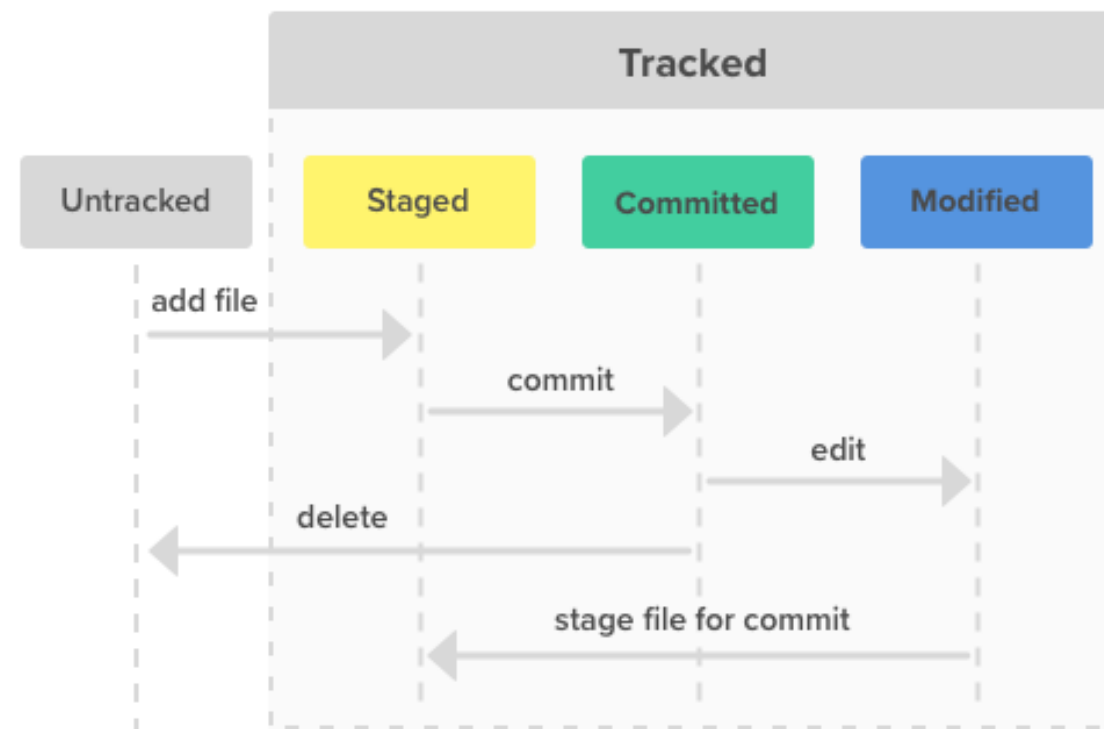
To take the files as they are in the staging area and store that snapshot permanently to your local git repo, use `git commit`

Use `git checkout` to pull any previous version from the local repo into the working tree



LOCAL REPO: FILE STATES

- **Untracked:** files not tracked by Git
- **Staged:** means that you have marked a modified file in its current version to go into your next commit snapshot.
- **Committed:** means that the data is safely stored in your local database.
- **Modified:** means that you have changed the file but have not committed it to your database yet.





DON'T COMMIT GENERATED FILES OR DEPENDENCIES

add a `.gitignore` file in your repository's root to automatically tell Git which files or paths you don't want to track.

```
# Compiled class file
*.class

# Log file
*.log

# BlueJ files
*.ctxt

# Mobile Tools for Java (J2ME)
.mtj.tmp/

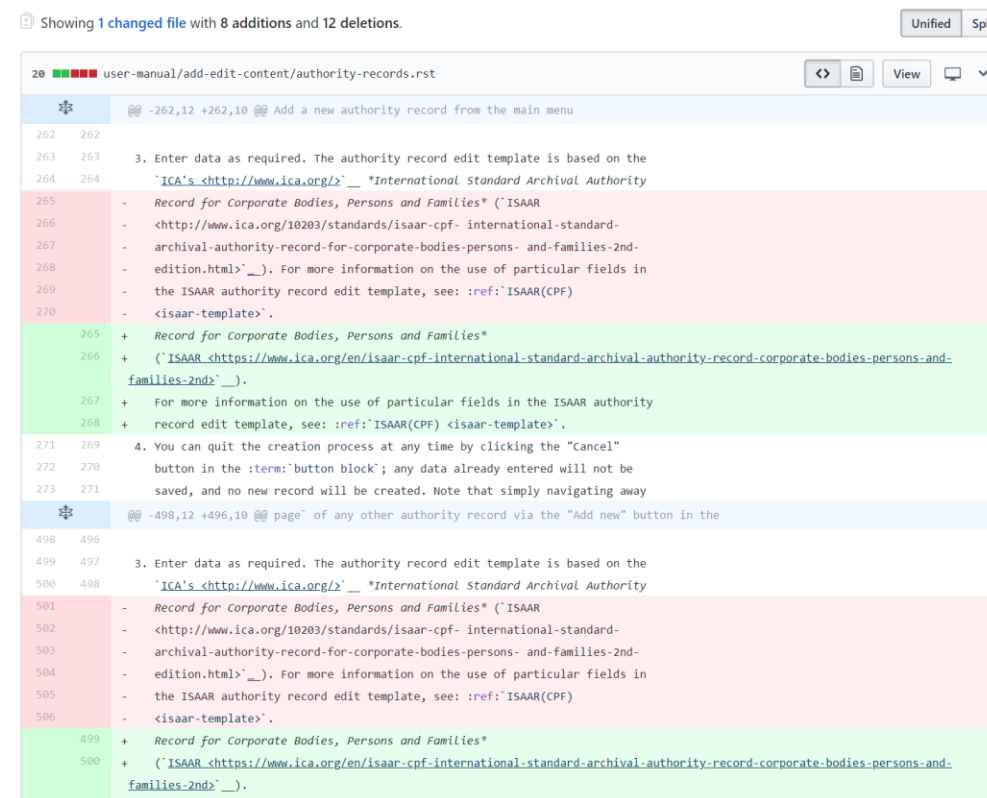
# Package Files #
*.jar
*.war
*.nar
*.ear
*.zip
*.tar.gz
*.rar

# virtual machine crash logs, see http://www.java.com/en/download/help/error_hotspot.xml
hs_err_pid*
```

GIT BEST PRACTICES

Make clean, single-purpose commits

- A commit should address a single, atomic issue (e.g., a bug fix), instead of addressing many issues (e.g., a bug fix + a new feature + refactorings +)
- Small code changes are easy to digest
 - 35% of the changes at Google are to a single file
- Small code changes allow quick approvals and quicker changes to the codebase
 - Most changes at Google are expected to be reviewed within a day





GIT BEST PRACTICES

Write meaningful commit messages

A good commit message should

- summarize the change
- explain what is changed and why
- e.g., “bug fix” is not a good change description

Commits on May 6, 2022

rename

 jendib committed on May 6, 2022

Commits on May 5, 2022

Allow the . (dot) and @ (at) character in usernames (sismics#637) ...

 hukoeth and Uli Koeth committed on May 5, 2022

Commits on Apr 17, 2022

Closes sismics#620: delete a non-existing document should return 404

 jendib committed on Apr 17, 2022

Closes sismics#632: validate POST /app/config_inbox and update docume... ...

 jendib committed on Apr 17, 2022

Add doc for search syntax (sismics#634)

 archiloque committed on Apr 17, 2022

Commits on Apr 15, 2022

Download zip of files not in same document (sismics#591)

 archiloque committed on Apr 15, 2022

WORKING WITH REMOTE REPO

To collaborate, you should first “link” a local repo and a remote repo



```
git remote add <name> <URL>
```

```
origin  github.com/user_name/repo_name
```



WORKING WITH REMOTE REPO

To collaborate, you should first “link” a local repo and a remote repo



local
repository



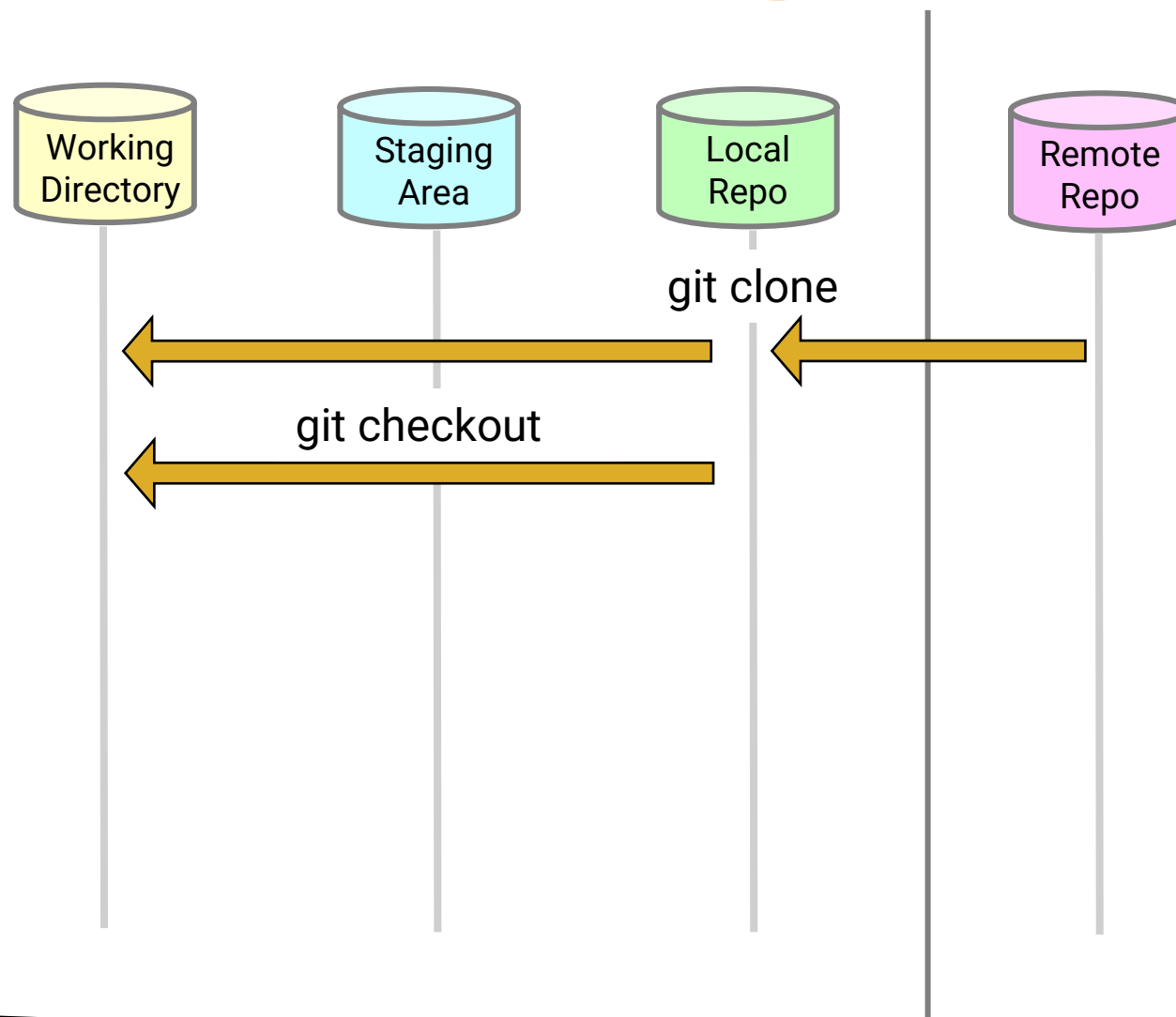
remote
repository

```
git clone <URL>
```



GIT CLONE

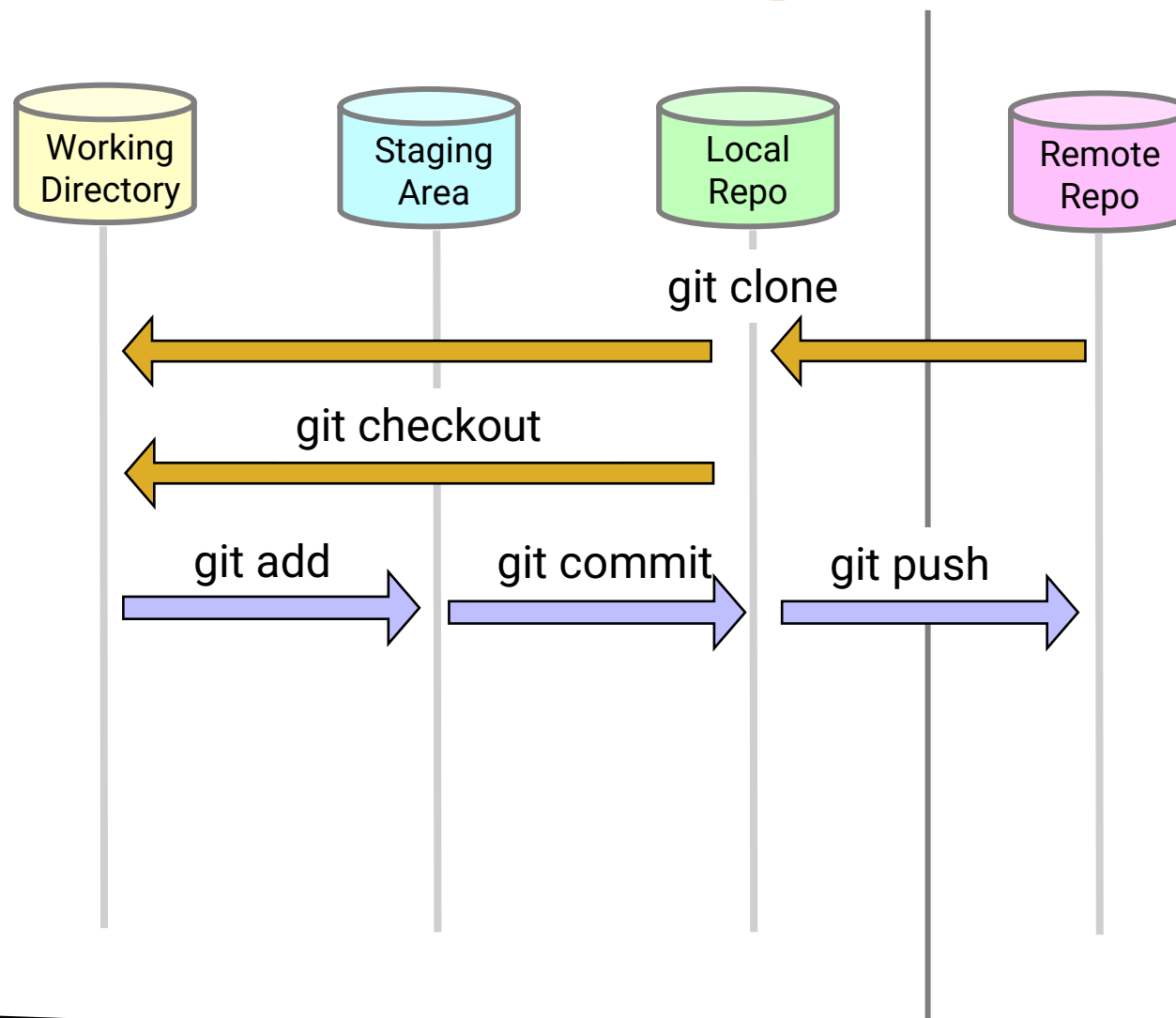
- Copies the entire remote repository into a new .git folder on your computer
- Automatically creates a remote connection named origin
- Checks out the latest version into the working directory so you can start right away





PUSH TO REMOTE REPO

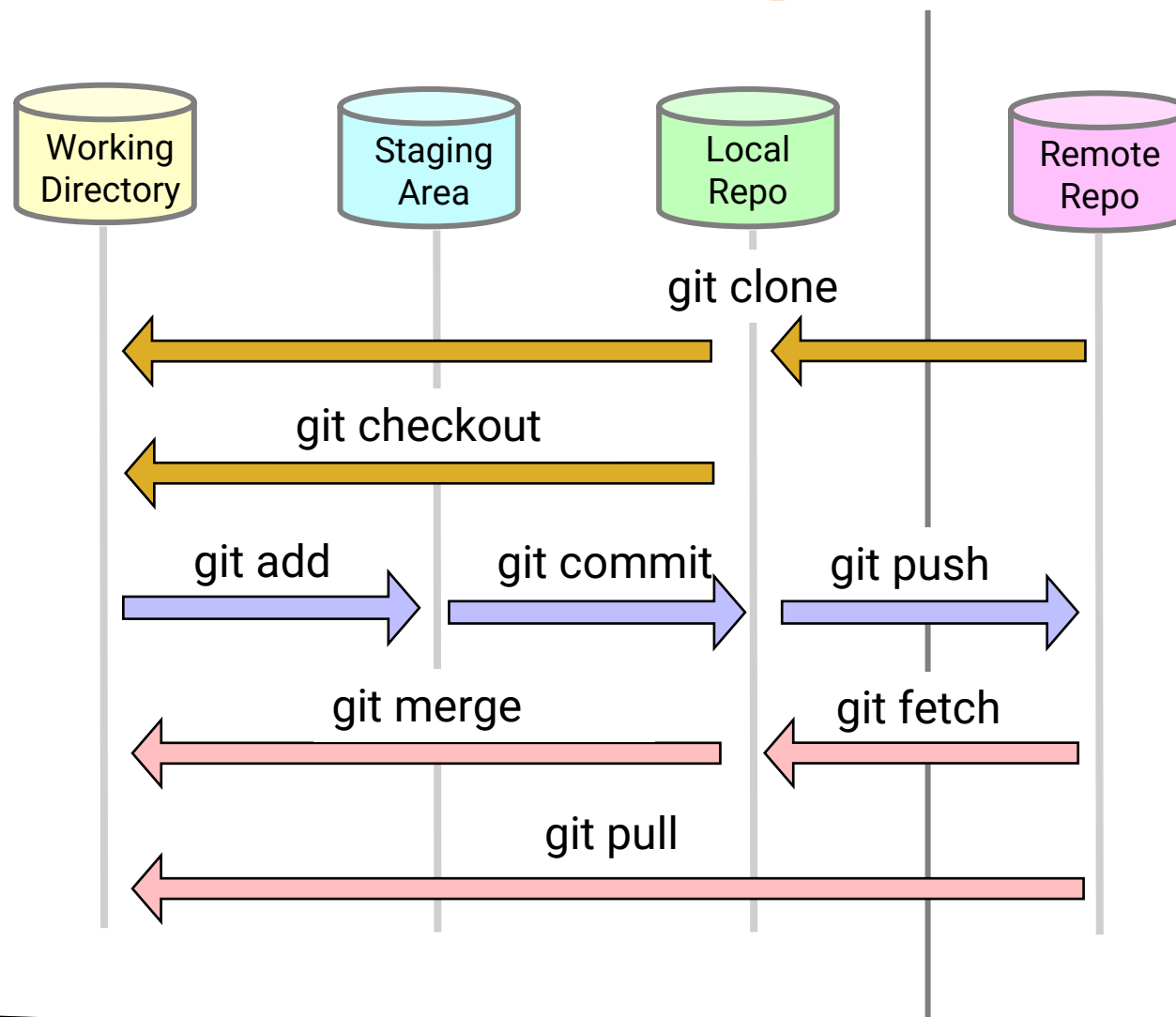
- When you have your project at a point that you want to share, you have to push it to the remote
- Command
`git push <remote> <local>`
- where `origin` is the default remote name and `main` is the default local branch name

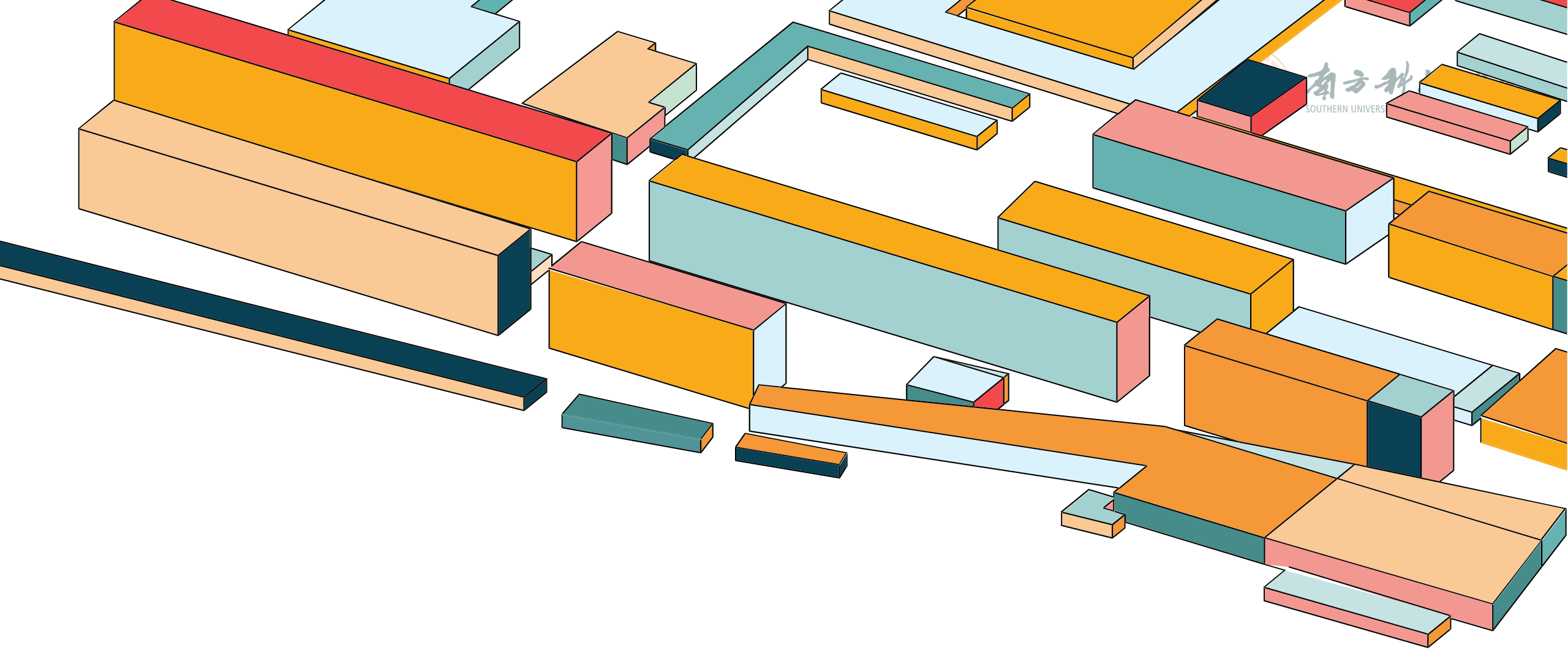




PULL FROM REMOTE REPO

- `git fetch <remote>` fetches any new work that has been pushed to that server since you cloned (or last fetched from) it
- `git fetch` only downloads the data to your local repository, it doesn't automatically merge it with your working directory. You have to `git merge` it manually into your work when you're ready.
- `git pull <remote>` command automatically fetches and then merges to the working directory in one step





GIT INTERNALS



GIT INTERNALS

- At the core of Git is a simple key-value data store.
 - Key: hash
 - Value: a sequence of bytes representing files, directories, commits, etc.
 - We can provide a value to git and git will calculate a unique key for it, which can be used later to retrieve the content.
- This key-value structure is persistent, i.e., it's stored in our disk, the **.git** directory
- It's impossible to change the contents of any file or directory without Git knowing about it.



GIT INTERNALS

- To understand the core of Git internals, there are 3 things to we should know
 - **Objects:** content of files, directories, commits, and tags, identified by SHA-1 hash, stored in `.git/objects/`
 - **References:** A branch, remote branch or a tag, which is simply a pointer to an object, stored in plain text in `.git/refs/`
 - **Index:** a staging area, stored as a binary file in `.git/index`.



TYPES OF GIT OBJECTS

- **Blob:** file content, identified by a hash
- Tree object
- Commit object
- Tag object

5b1d3..

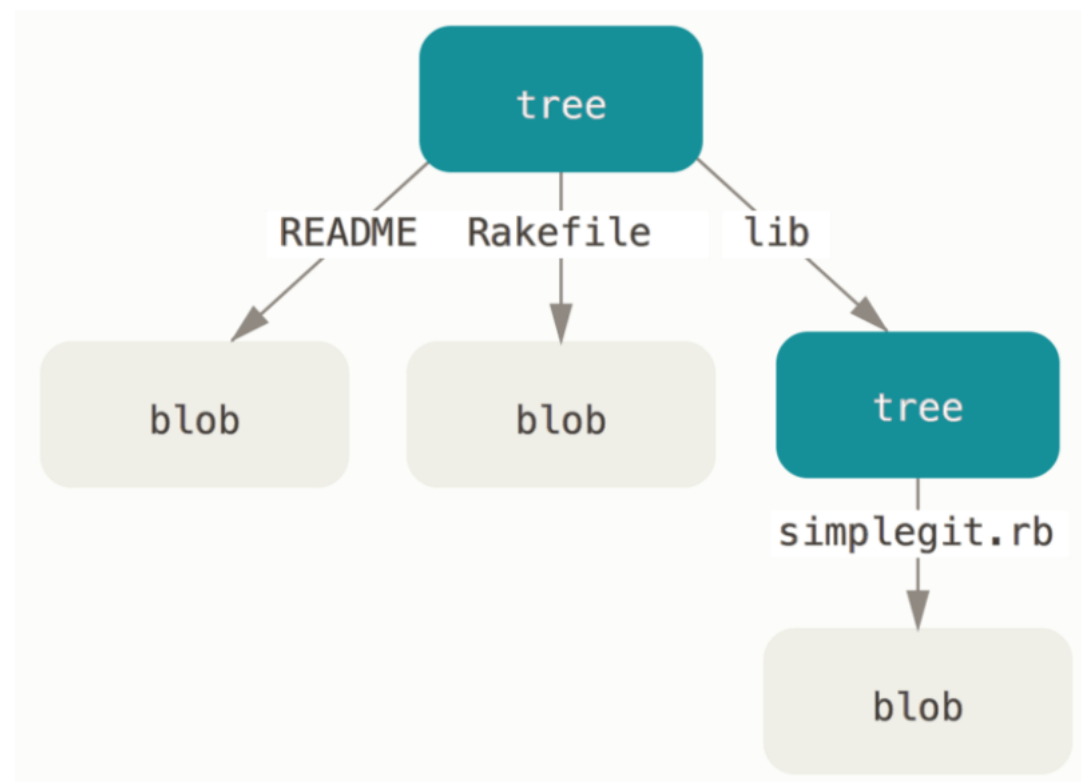
blob	size
<pre>#ifndef REVISION_H #define REVISION_H #include "parse-options.h" #define SEEN (1u<<0) #define UNINTERESTING (1u #define TREESAME (1u<<2)</pre>	

TYPES OF GIT OBJECTS

- Blob: file content, identified by a hash
- **Tree object**: list of pointers to blob (file), or tree (directory), identified by a hash
- Commit object
- Tag object

c36d4..

tree		size
blob	5b1d3	README
tree	03e78	lib
tree	cdc8b	test
blob	cba0a	test.rb
blob	911e7	xdiff





TYPES OF GIT OBJECTS

- Blob: file content, identified by a hash
- Tree object: list of pointers to blob (file), or tree (directory), identified by a hash
- **Commit object:**
 - Reference to the top-level tree for the snapshot of the project at that point
 - Parent commits if any
 - Author info, commit message, etc.
- Tag object

ae668..

commit		size
tree	c4ec5	
parent	a149e	
author	Scott	
committer	Scott	
my commit message goes here and it is really, really cool		



TYPES OF GIT OBJECTS

- Blob: file content, identified by a hash
- Tree object: list of pointers to blob (file), or tree (directory), identified by a hash
- Commit object:
 - Reference to the top-level tree for the snapshot of the project at that point
 - Parent commits if any
 - Author info, commit message, etc.
- **Tag object**: name associated with a commit (+ potential metadata)

49e11..

tag		size
object	ae668	
type	commit	
tagger	Scott	
my tag message that explains this tag		



EXAMPLE: GIT INTERNALS

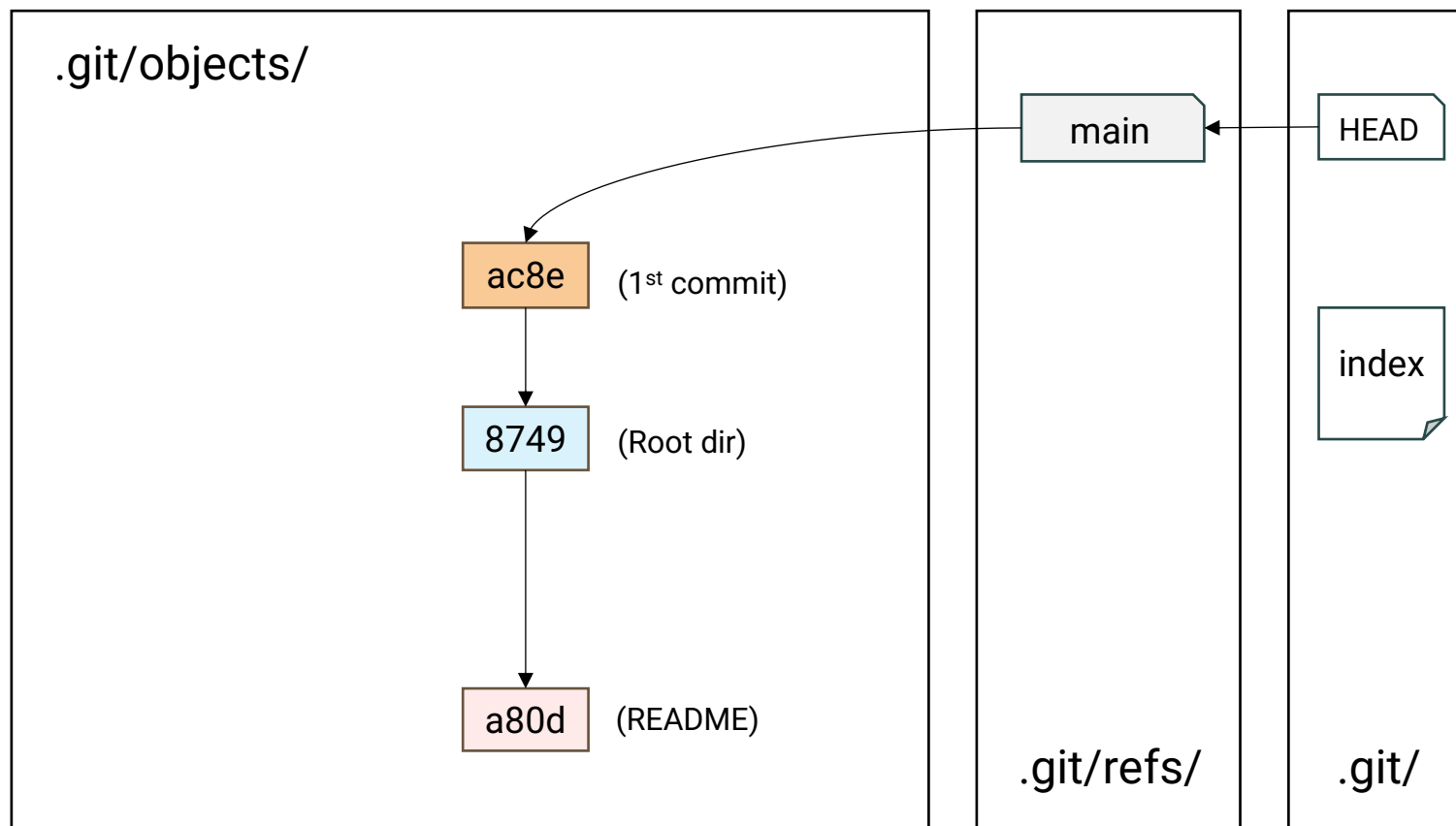
```
$ git init myproject
```



EXAMPLE: GIT INTERNALS

```
$ echo "Hello" >> README
```

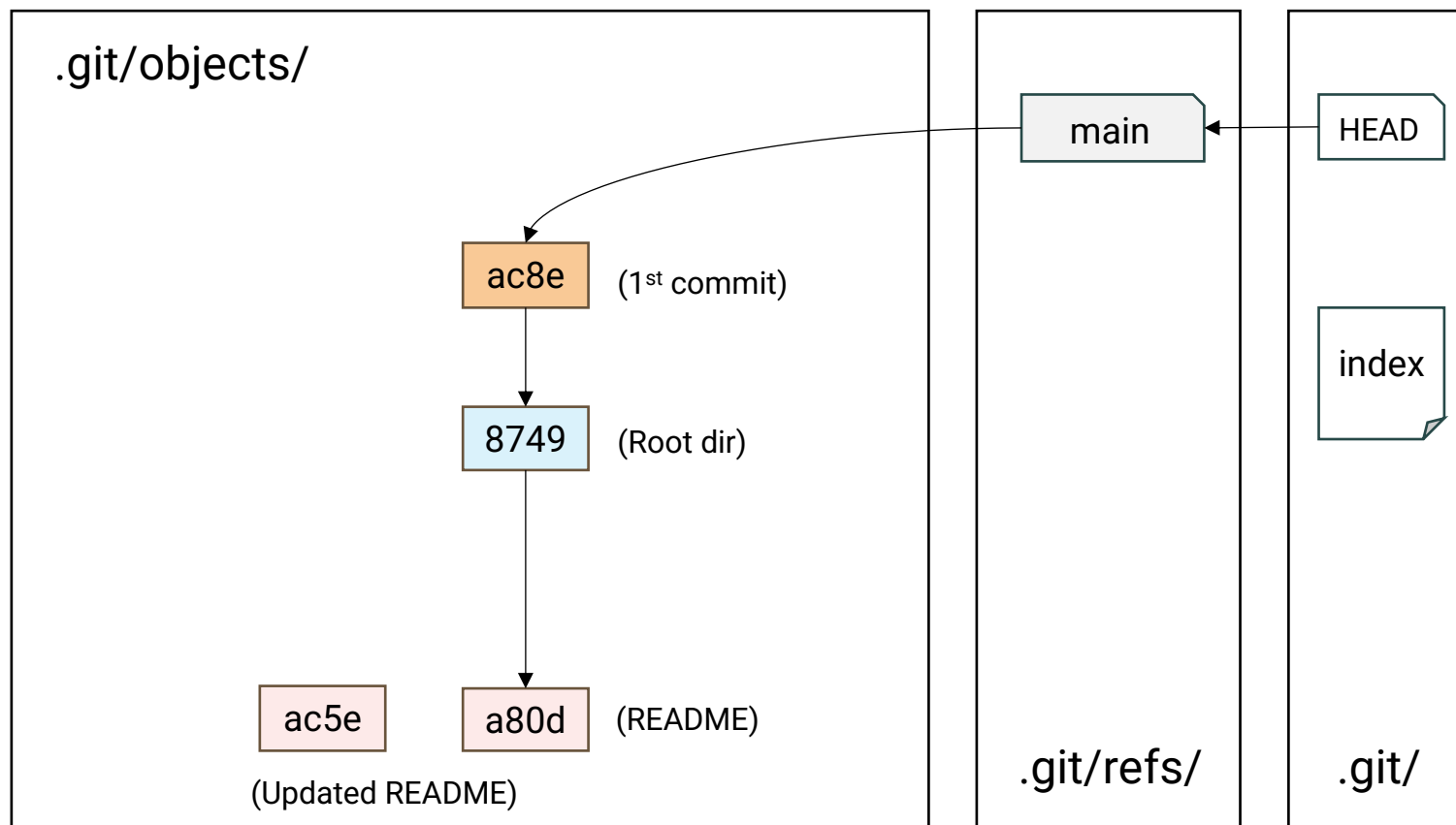
```
$ git add README
```



EXAMPLE: GIT INTERNALS

```
$ git commit -m "first commit"
```

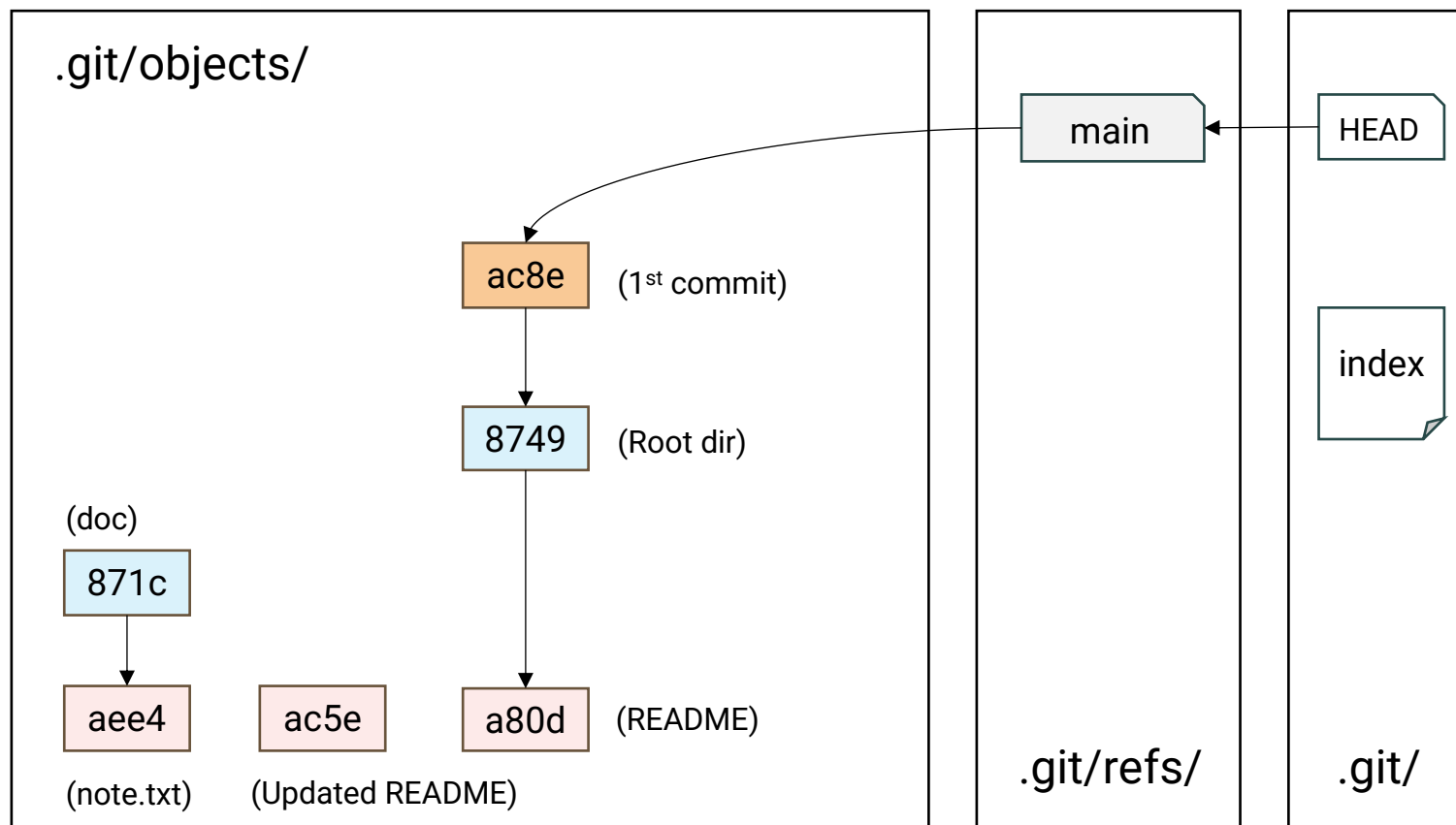
```
[main (root-commit) ac8e2c6]  
first commit  
1 file changed, 1 insertion(+)
```



EXAMPLE: GIT INTERNALS

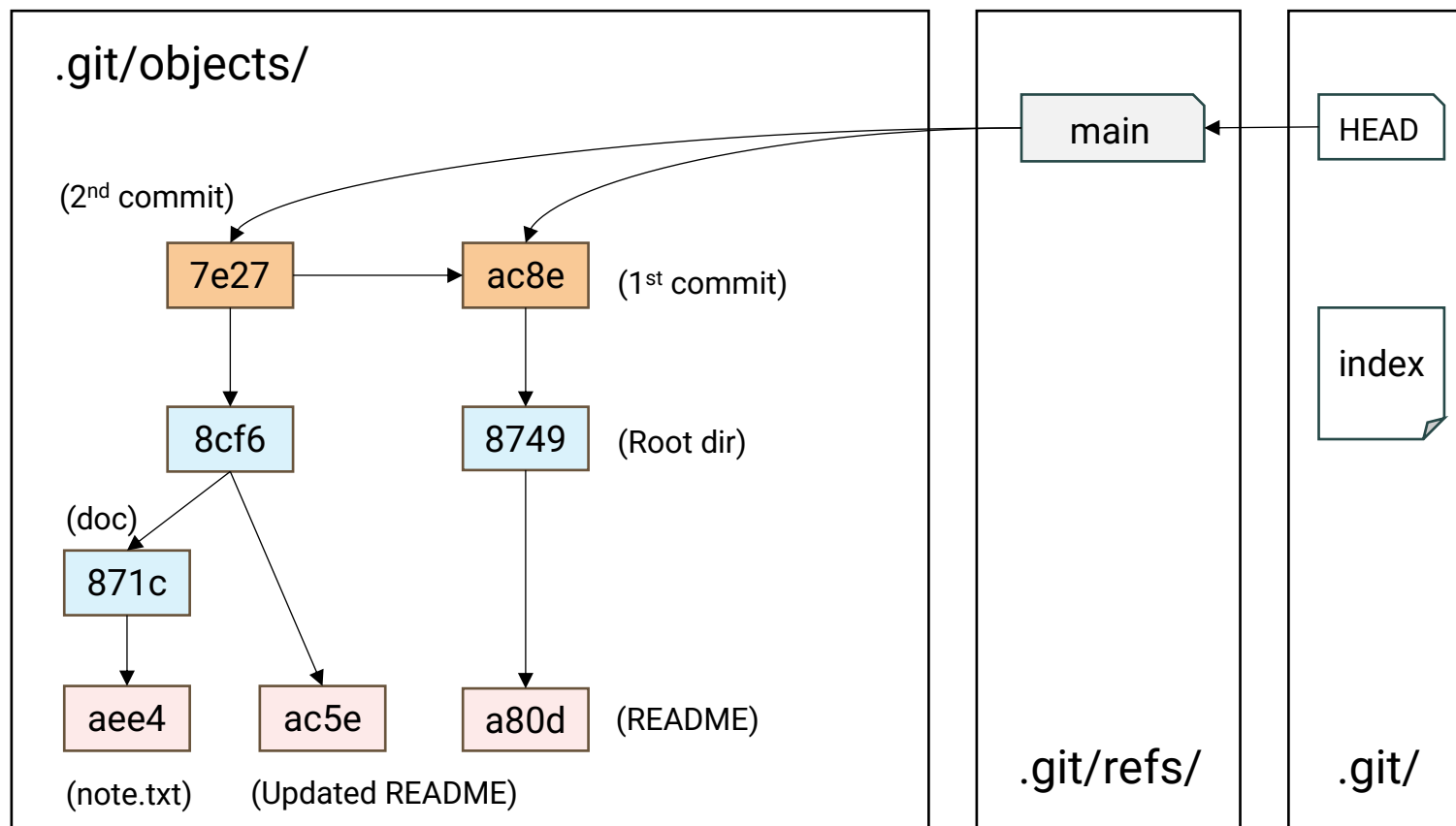
```
$ echo "Hi!!!" >> README
```

```
$ git add README
```

EXAMPLE: GIT INTERNALS

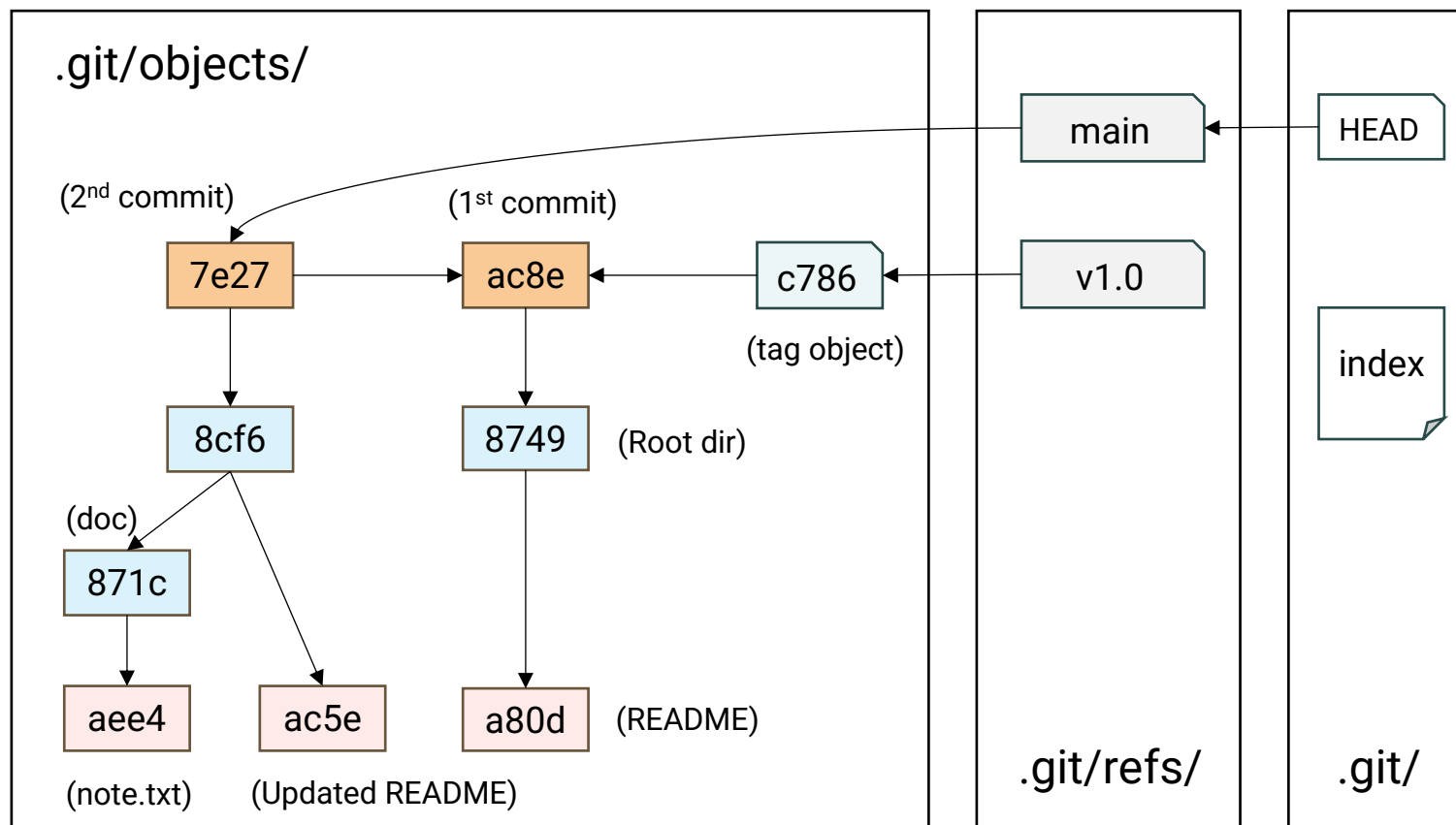
```
$ mkdir doc  
$ echo "note" >> doc/note.txt  
$ git add doc
```



EXAMPLE: GIT INTERNALS

```
$ git commit -m "second commit"

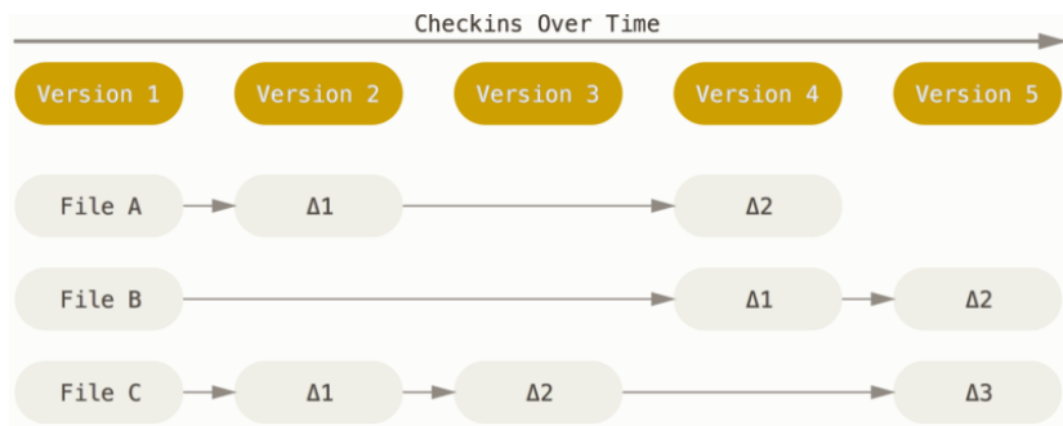
[main 7e27295] second commit
2 files changed, 2 insertions(+)
```



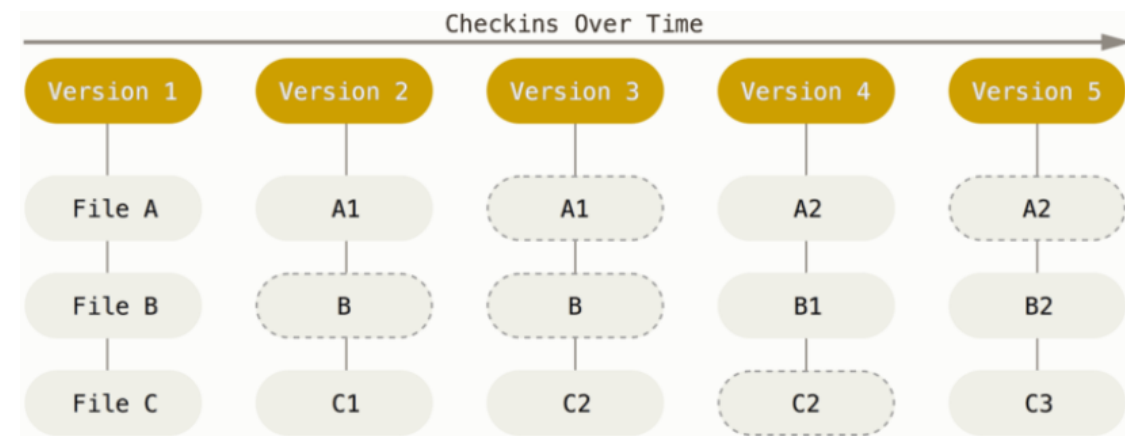
EXAMPLE: GIT INTERNALS

```
$ git tag v1.0 ac8e
```

SNAPSHOTS, NOT DIFFERENCES



Delta-based VCS (e.g., CVS, SVN)

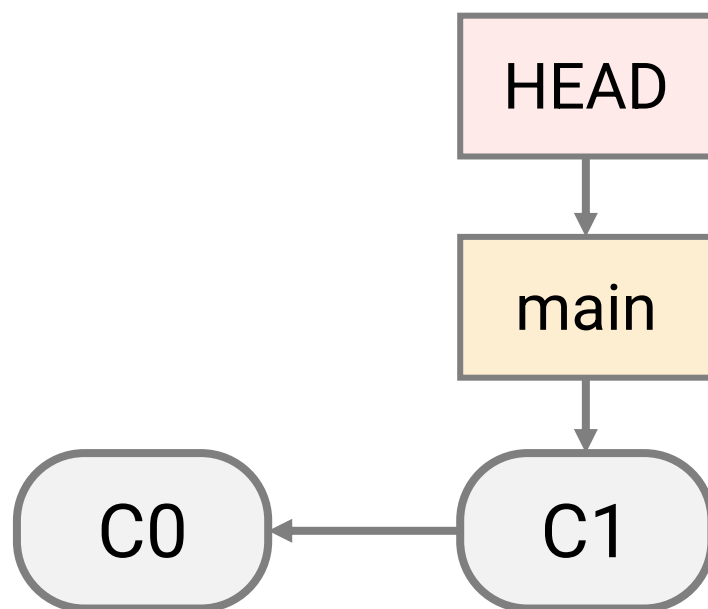


Snapshot-based VCS (e.g., git)



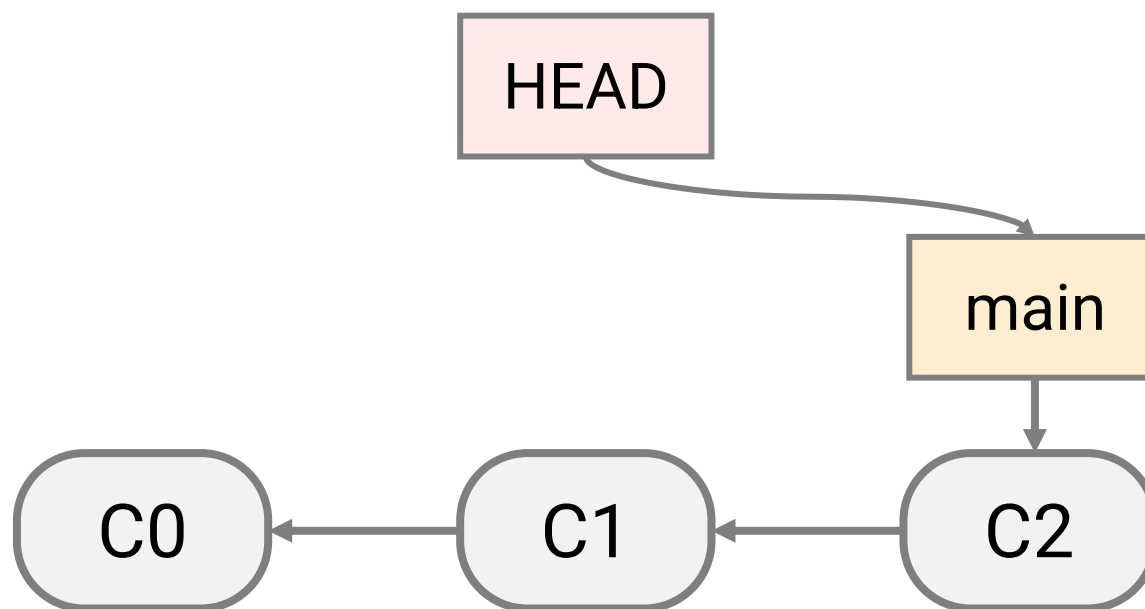
GIT BRANCHING

- “Killer feature” of Git
- Branching means you diverge from the main line of development and continue to do work without messing with that main line
- Git branching is incredibly lightweight: making branches and switching back and forth between branches are nearly instantaneous



COMMITTS AND POINTERS

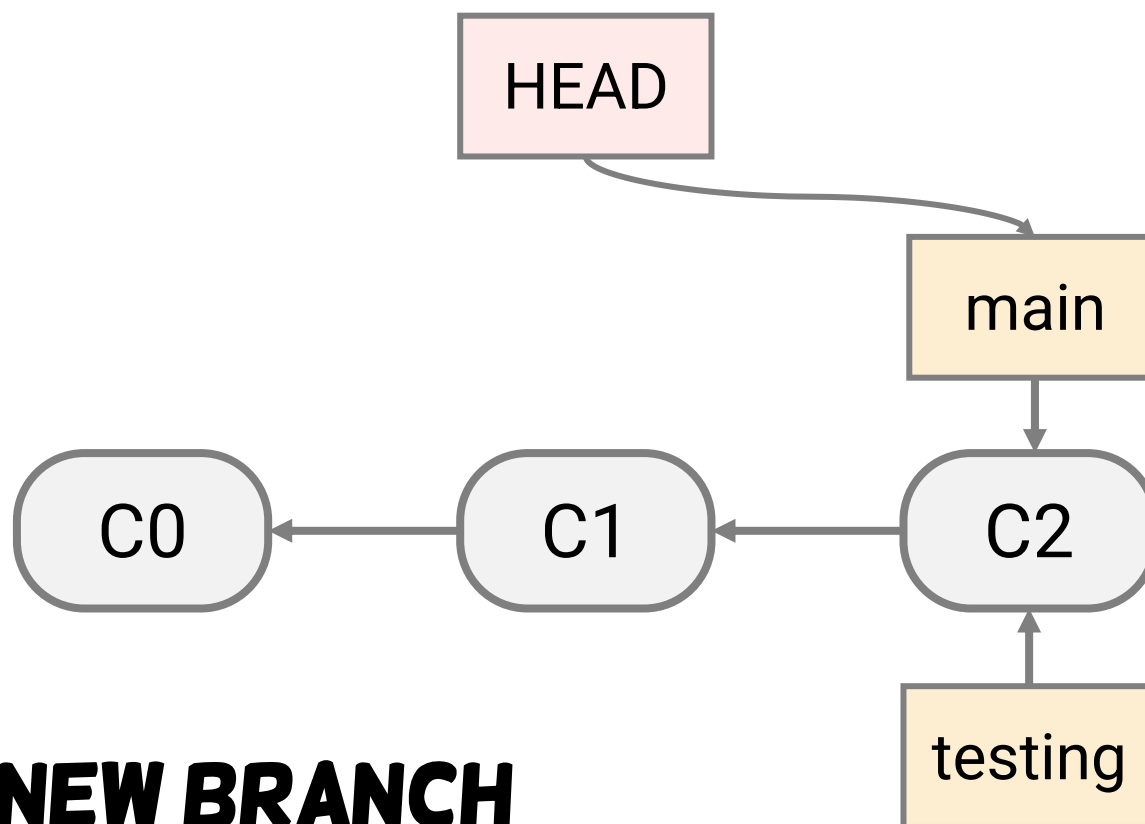
- Commit -> its parent
- main -> latest commit
- HEAD -> current location



COMMITTS AND POINTERS

- Commit -> its parent
- main -> latest commit
- HEAD -> current location

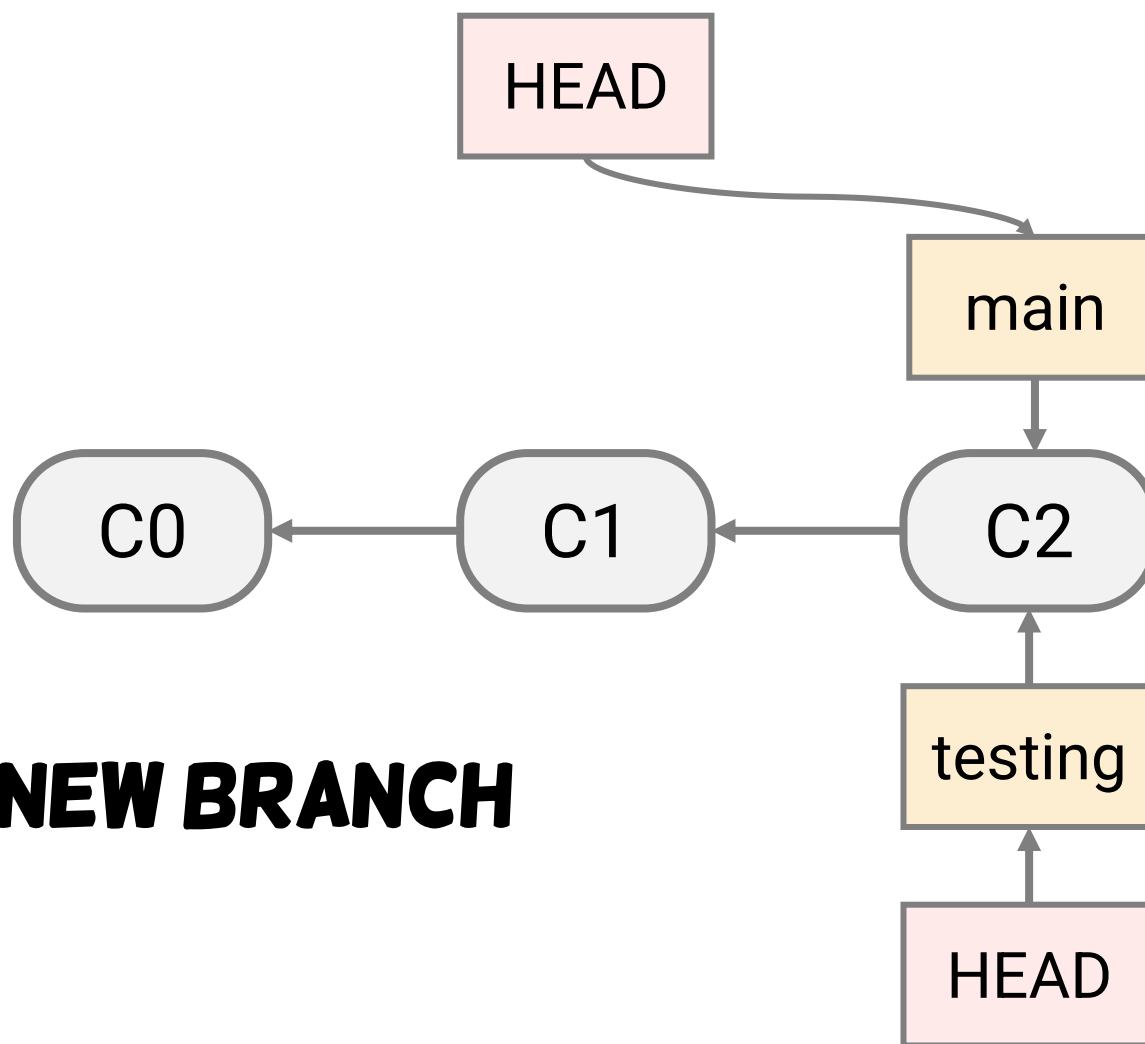
Every time you commit, the main branch pointer moves forward automatically.



`git branch <branch>` : create a new pointer to the **same commit** you're currently on

The `git branch` command only created a new branch: **it didn't switch to that branch.**

CREATE NEW BRANCH



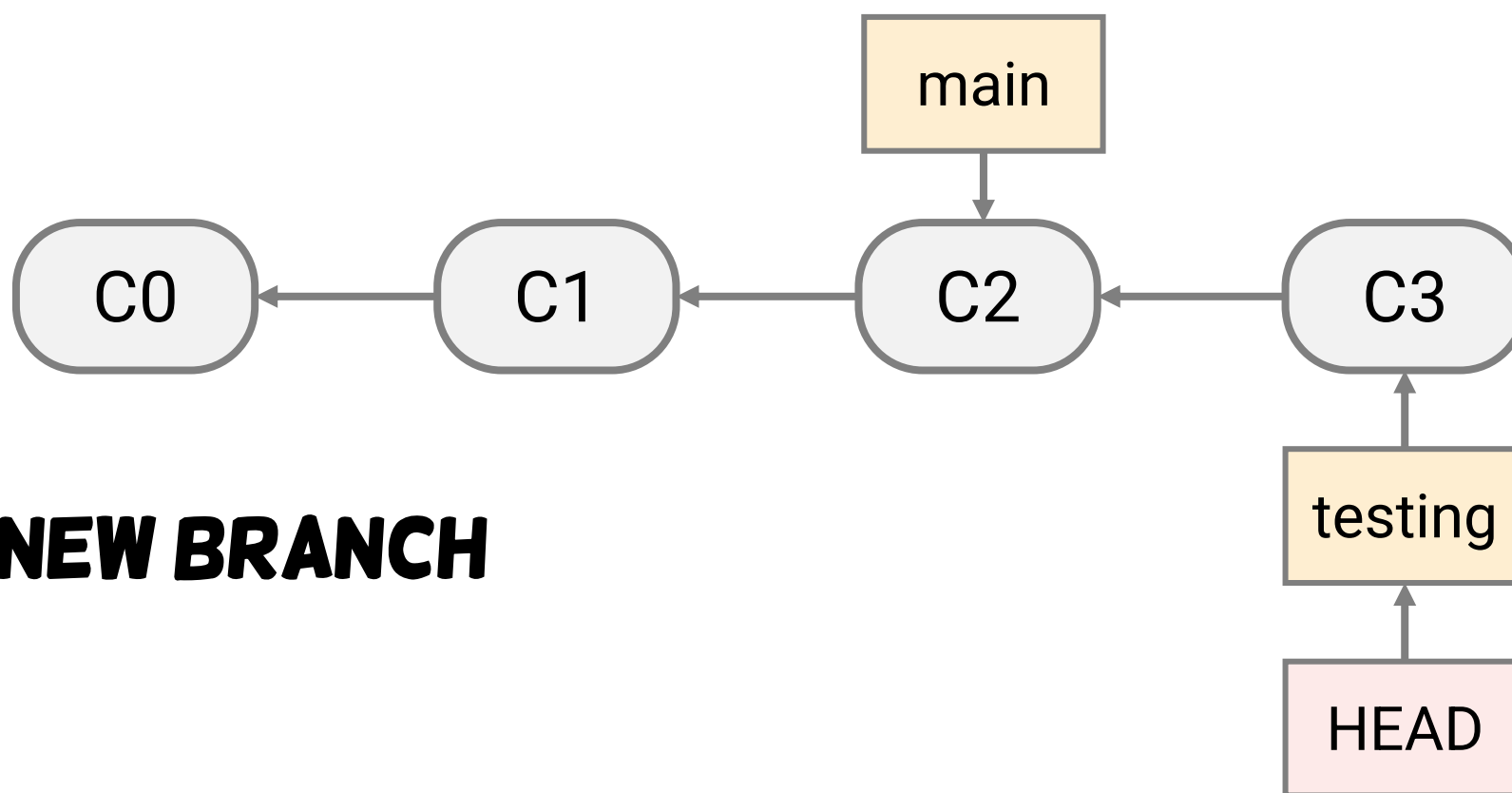
`git checkout <branch>`:
switch branch, which moves
HEAD to point to <branch>

`git checkout -b <branch>`:
Shortcut to create a branch and
switch to it.

CREATE NEW BRANCH



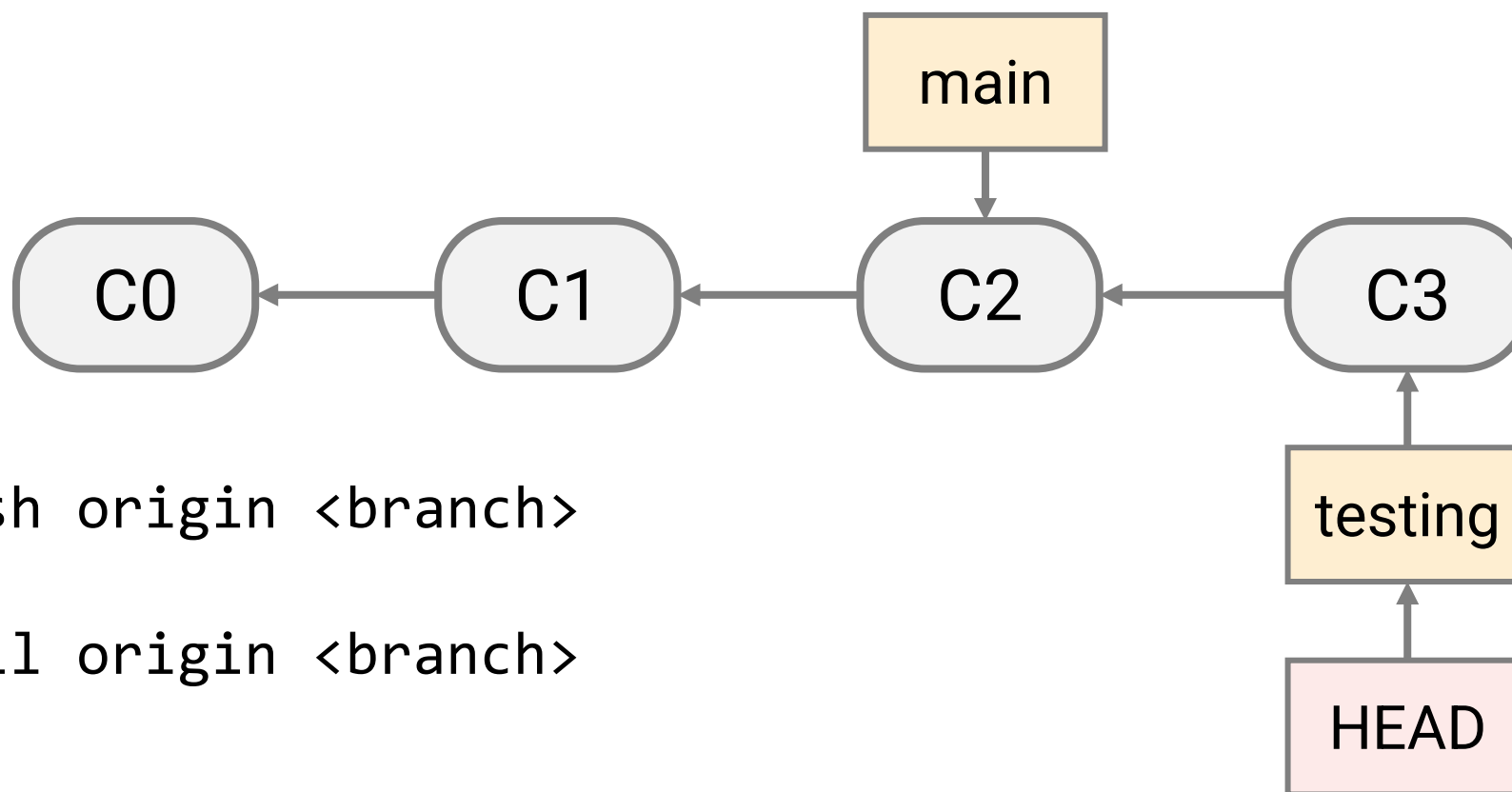
From here, making a new commit will make the testing branch move forward, while the main branch stays the same



CREATE NEW BRANCH

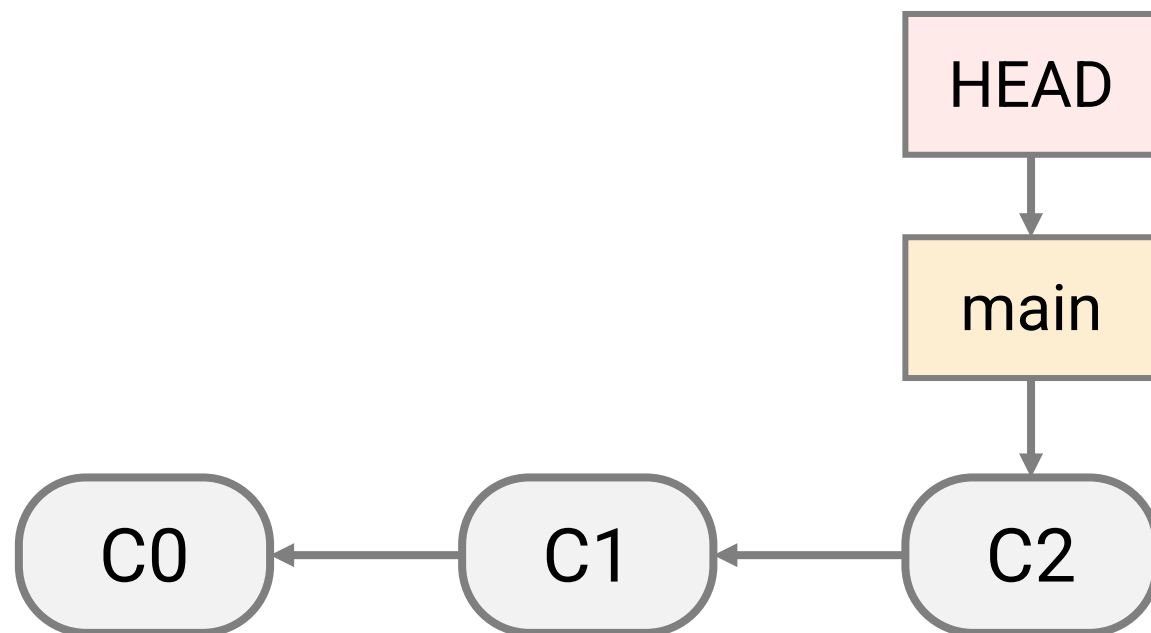


SHARING BRANCHES

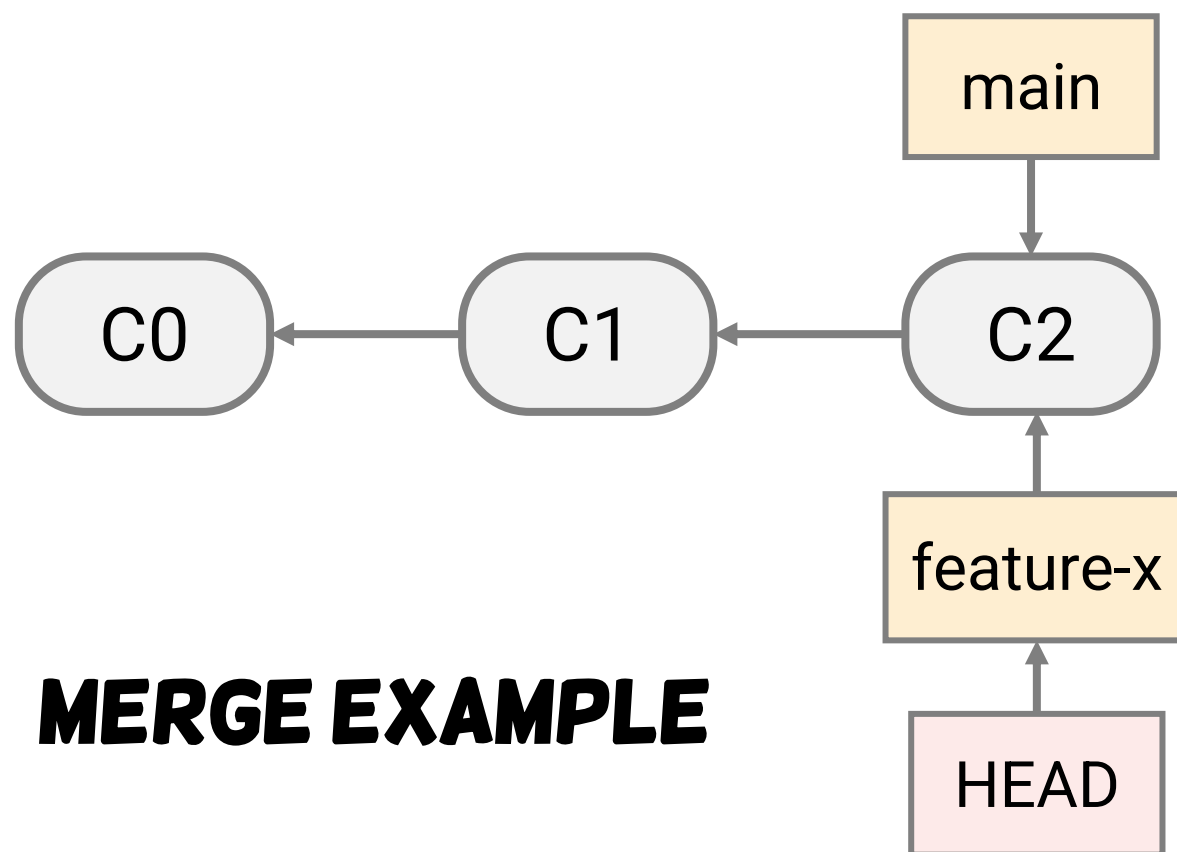


```
$ git push origin <branch>
```

```
$ git pull origin <branch>
```

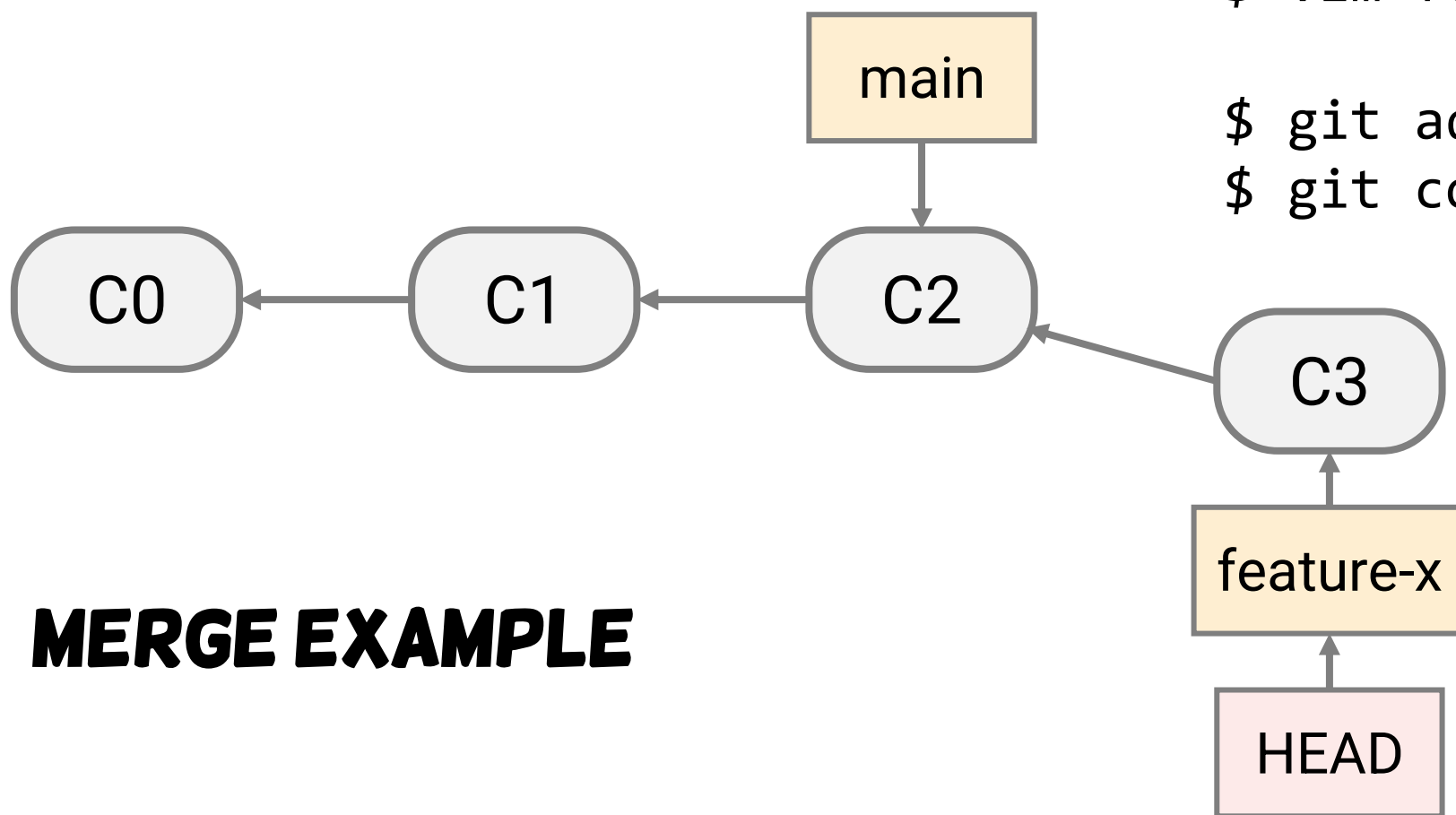


MERGE EXAMPLE



MERGE EXAMPLE

```
$ git checkout -b feature-x
```

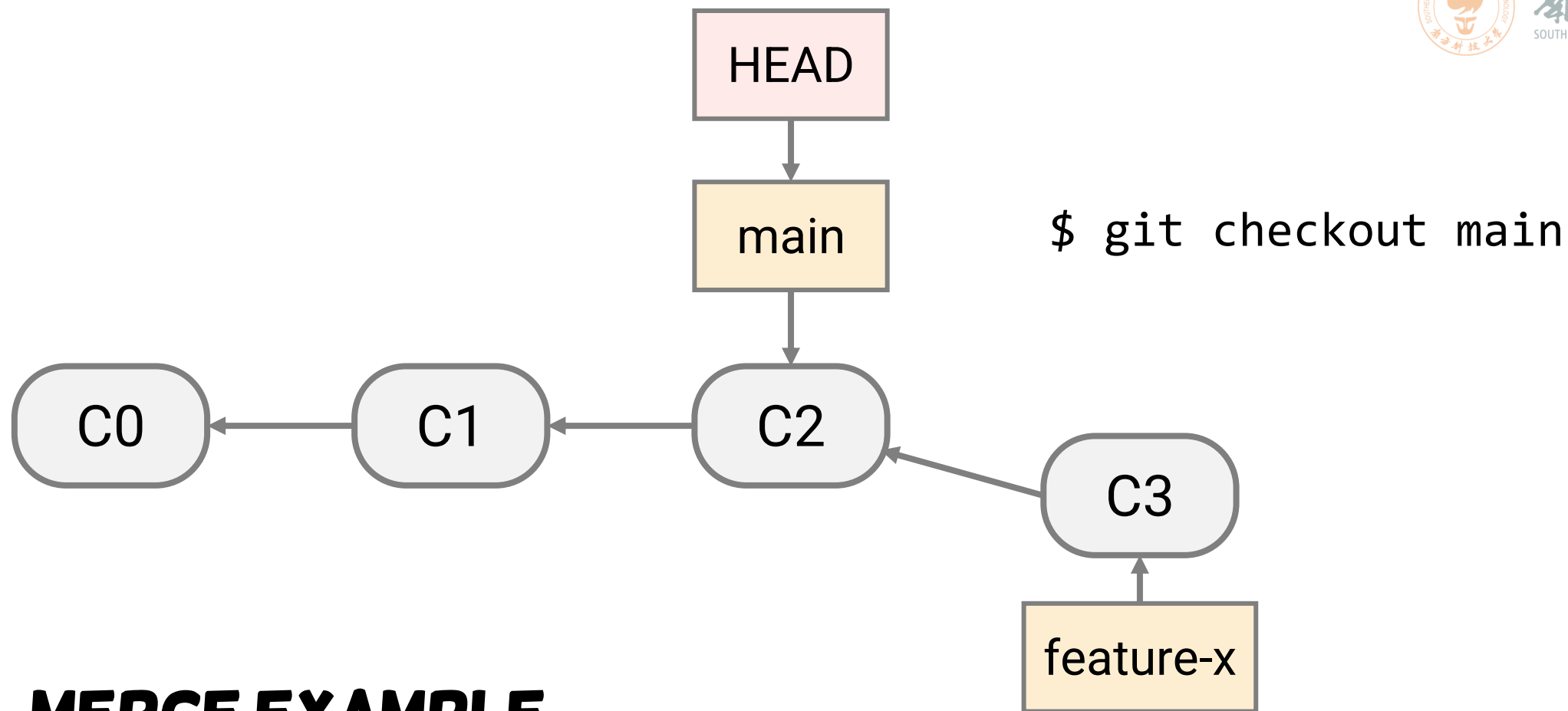


```
$ vim feature.html
```

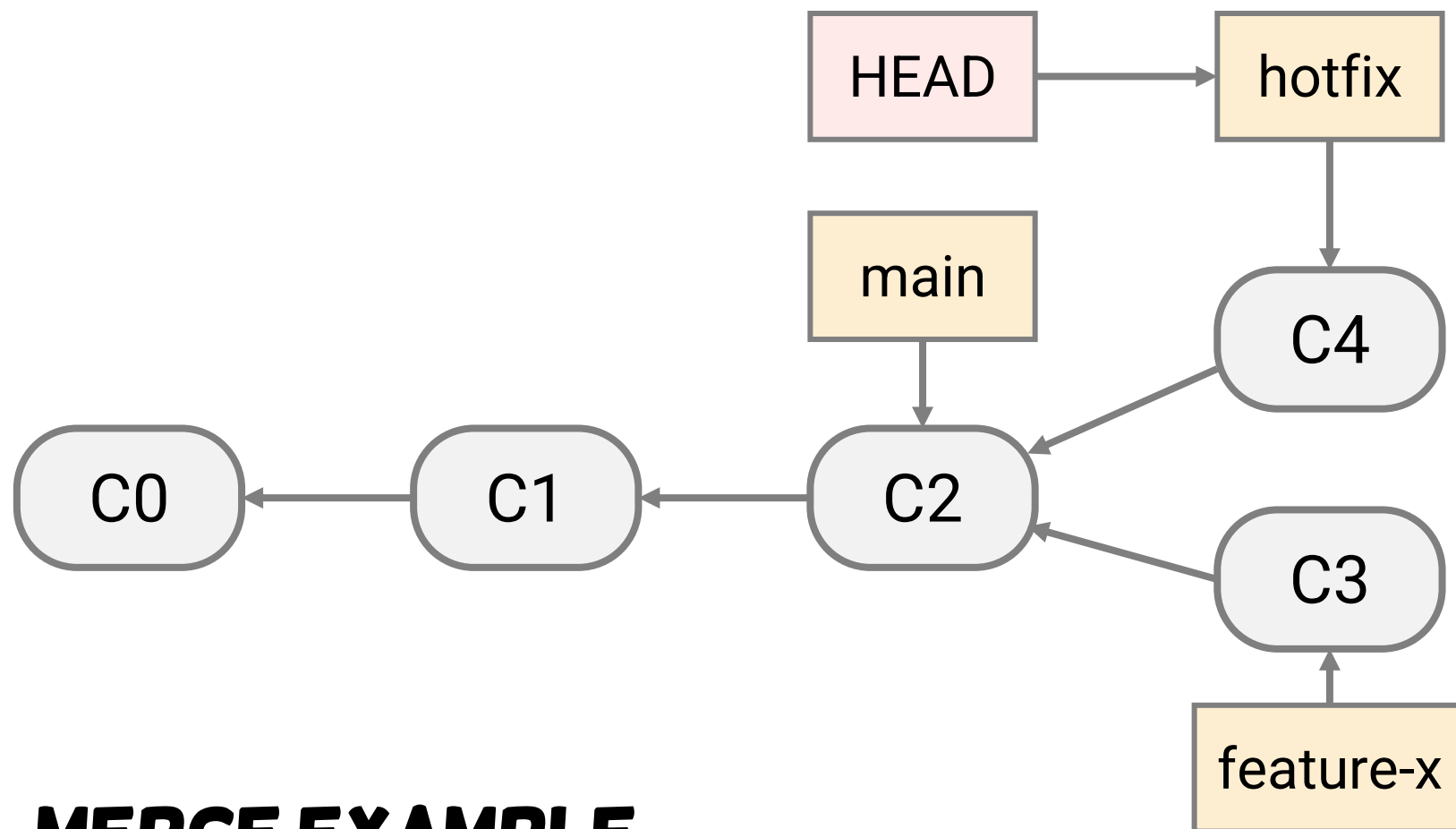
```
$ git add feature.html
```

```
$ git commit -m "my feature"
```

MERGE EXAMPLE



MERGE EXAMPLE



MERGE EXAMPLE

```
$ git checkout -b hotfix
```

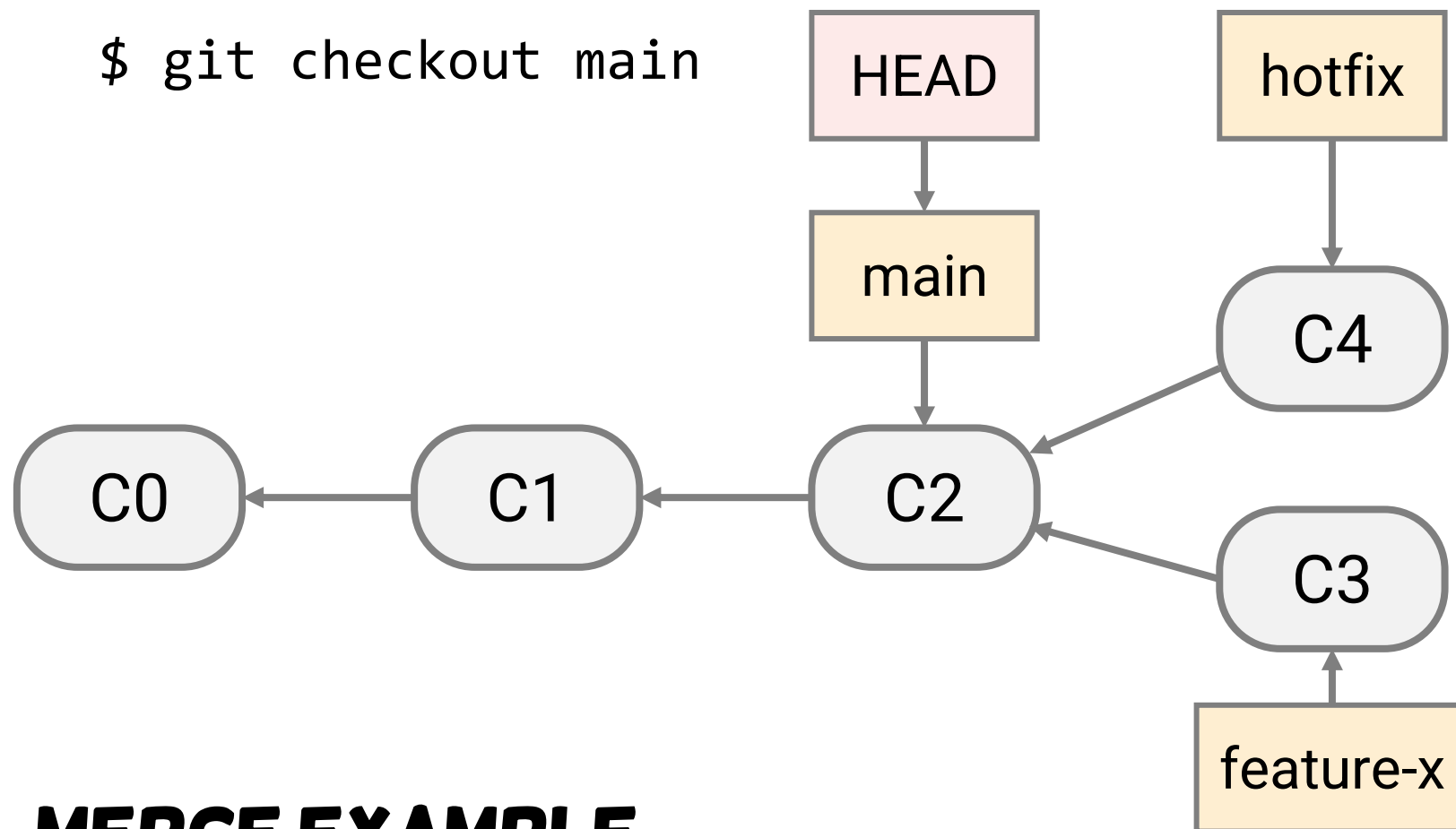
```
$ vim fix.py
```

```
$ git add fix.py
```

```
$ git commit -m "fix"
```




\$ git checkout main



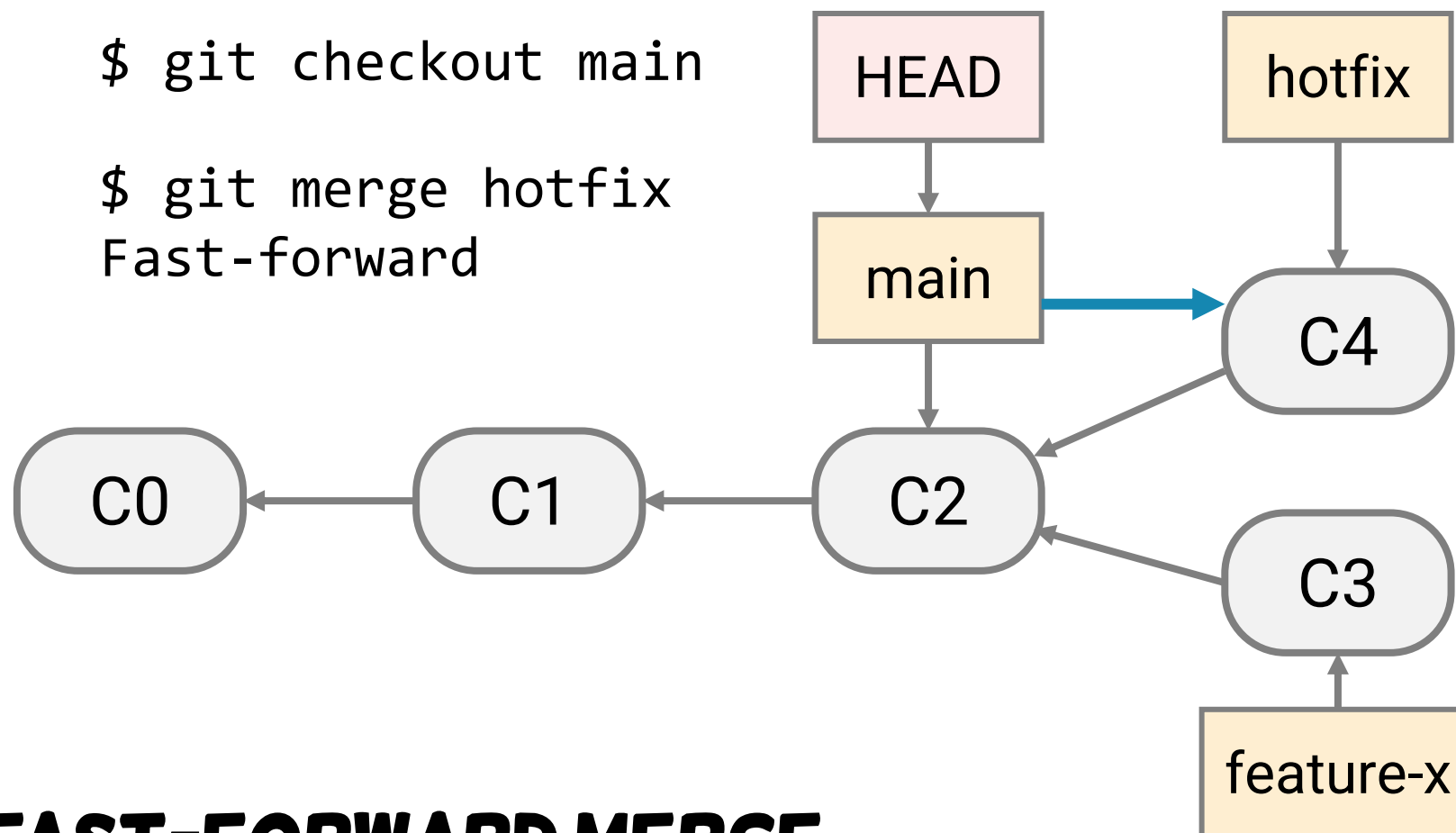
MERGE EXAMPLE



```
$ git checkout main
```

```
$ git merge hotfix
```

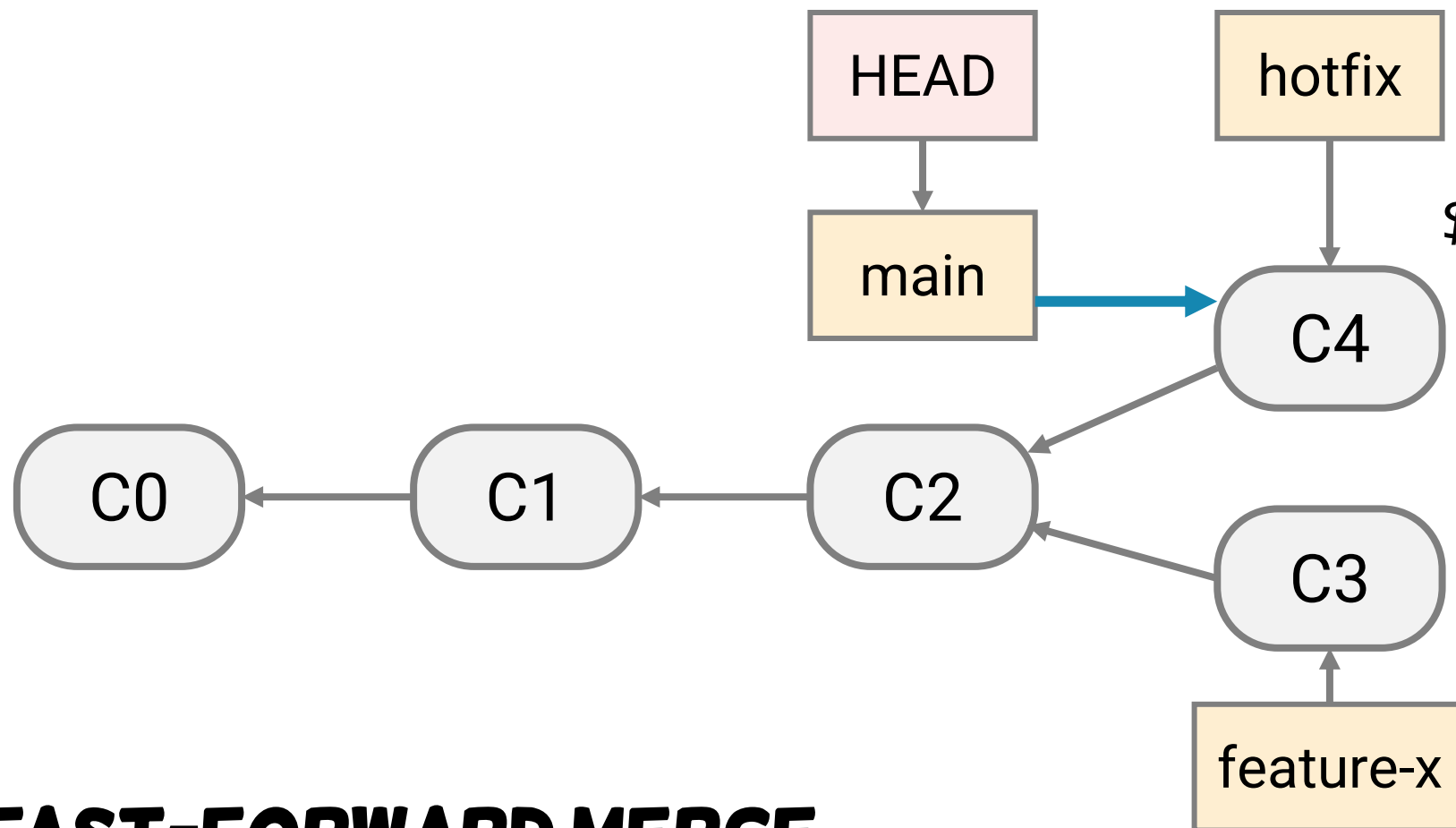
Fast-forward



C4 pointed to by the branch hotfix you merged in was directly ahead of C2 you're on

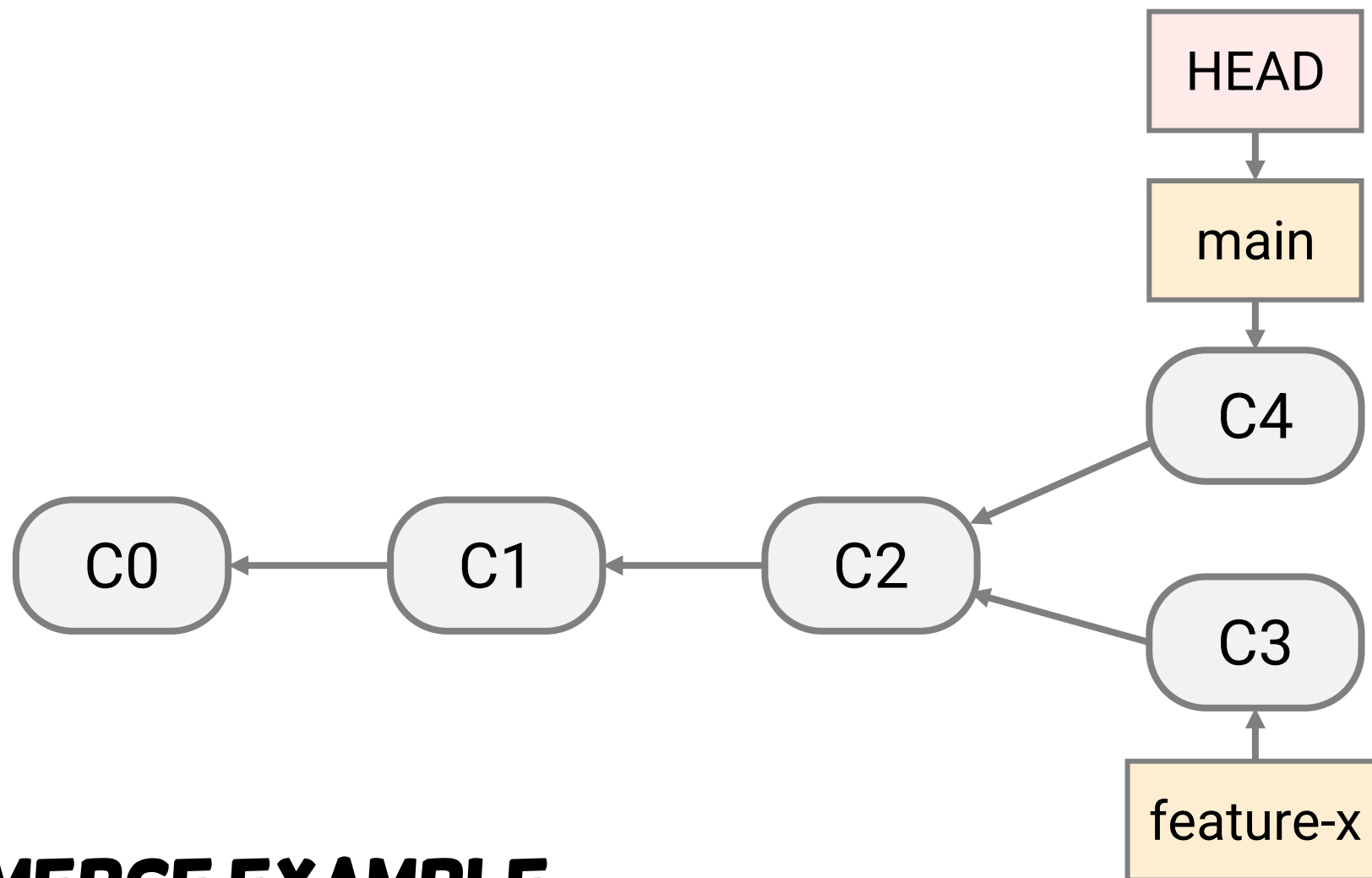
Git simply moves the pointer forward, cuz there is no divergent work to merge

FAST-FORWARD MERGE

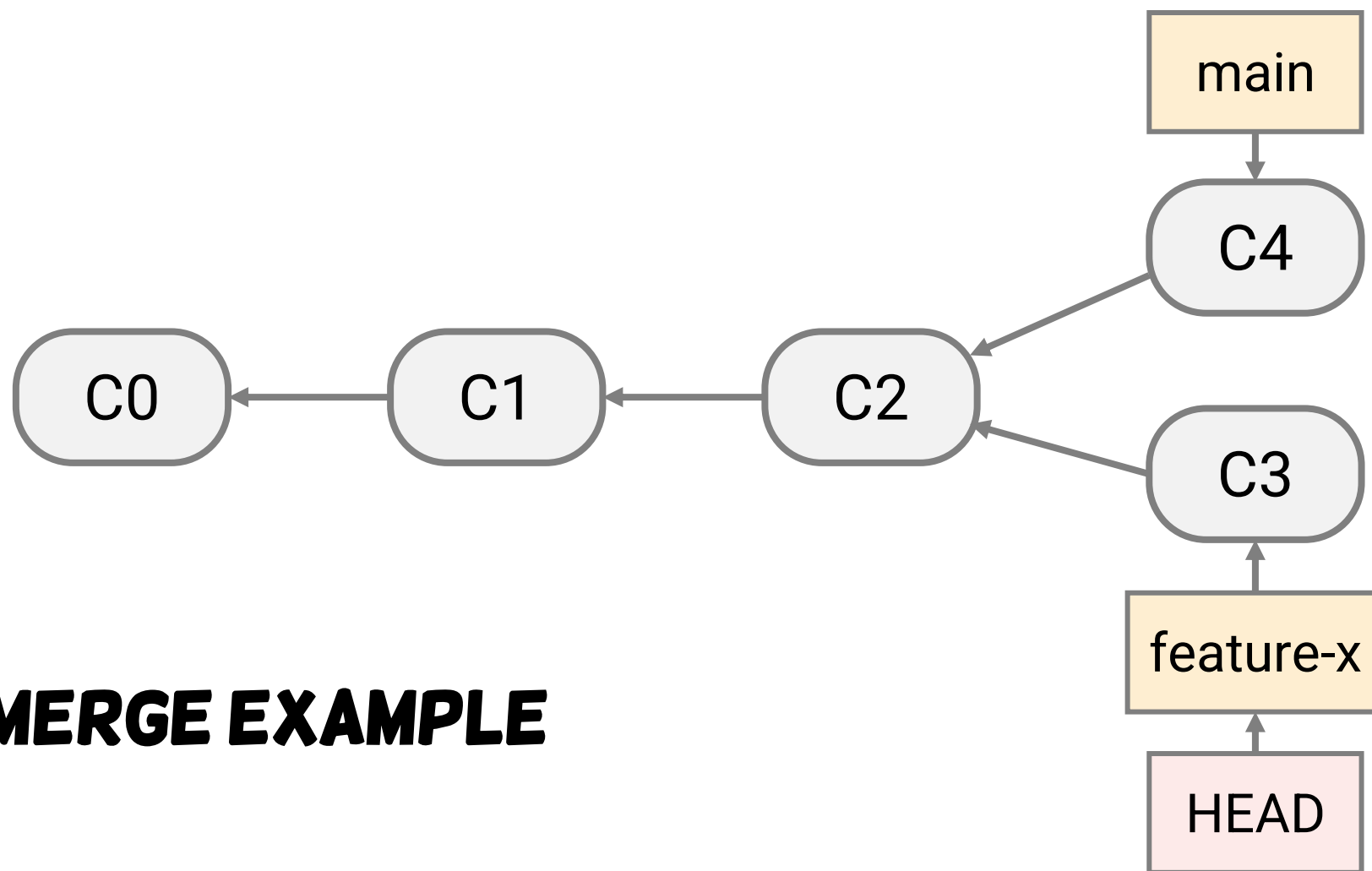


`$ git branch -d hotfix`

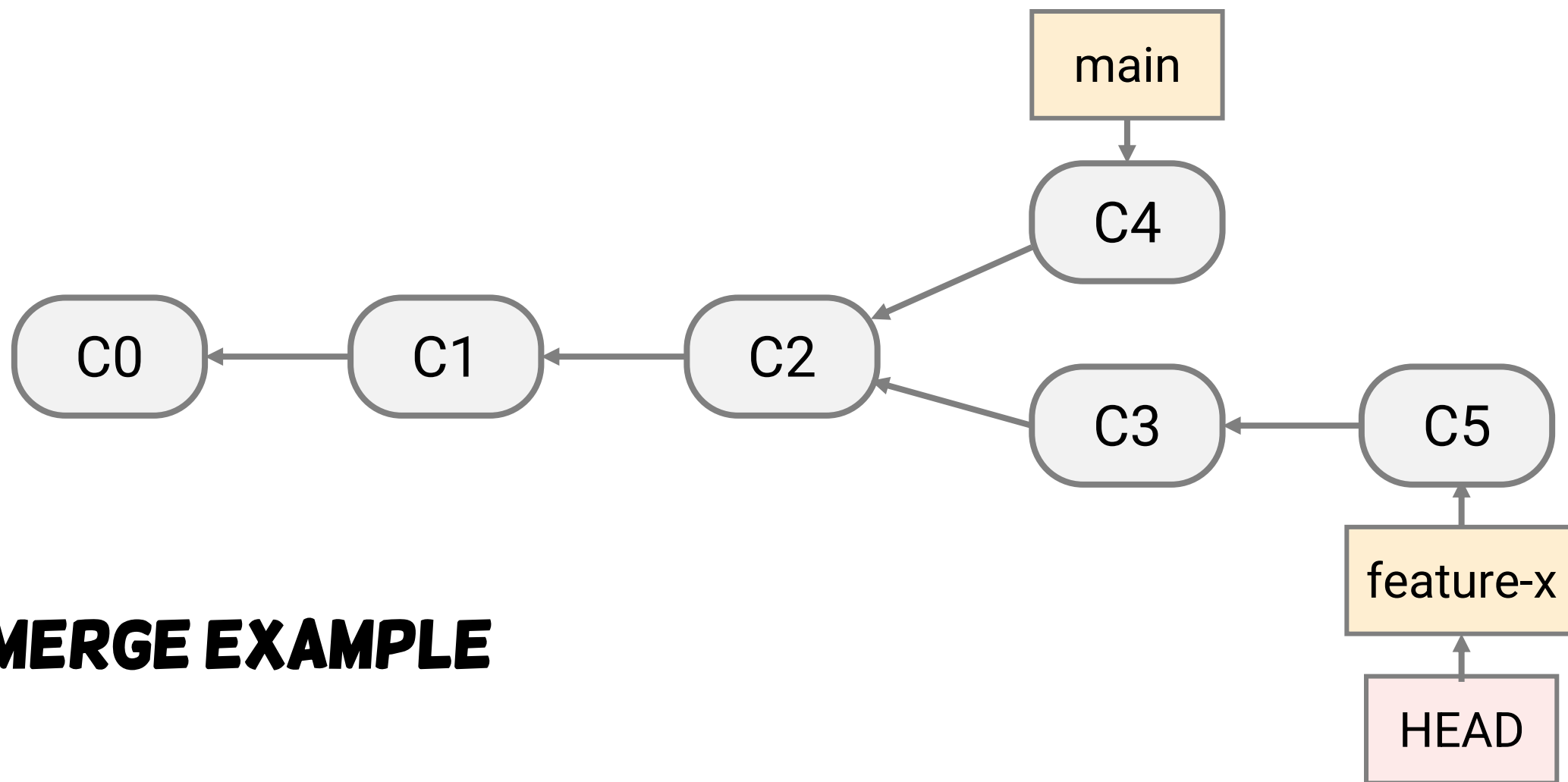
FAST-FORWARD MERGE



MERGE EXAMPLE



MERGE EXAMPLE

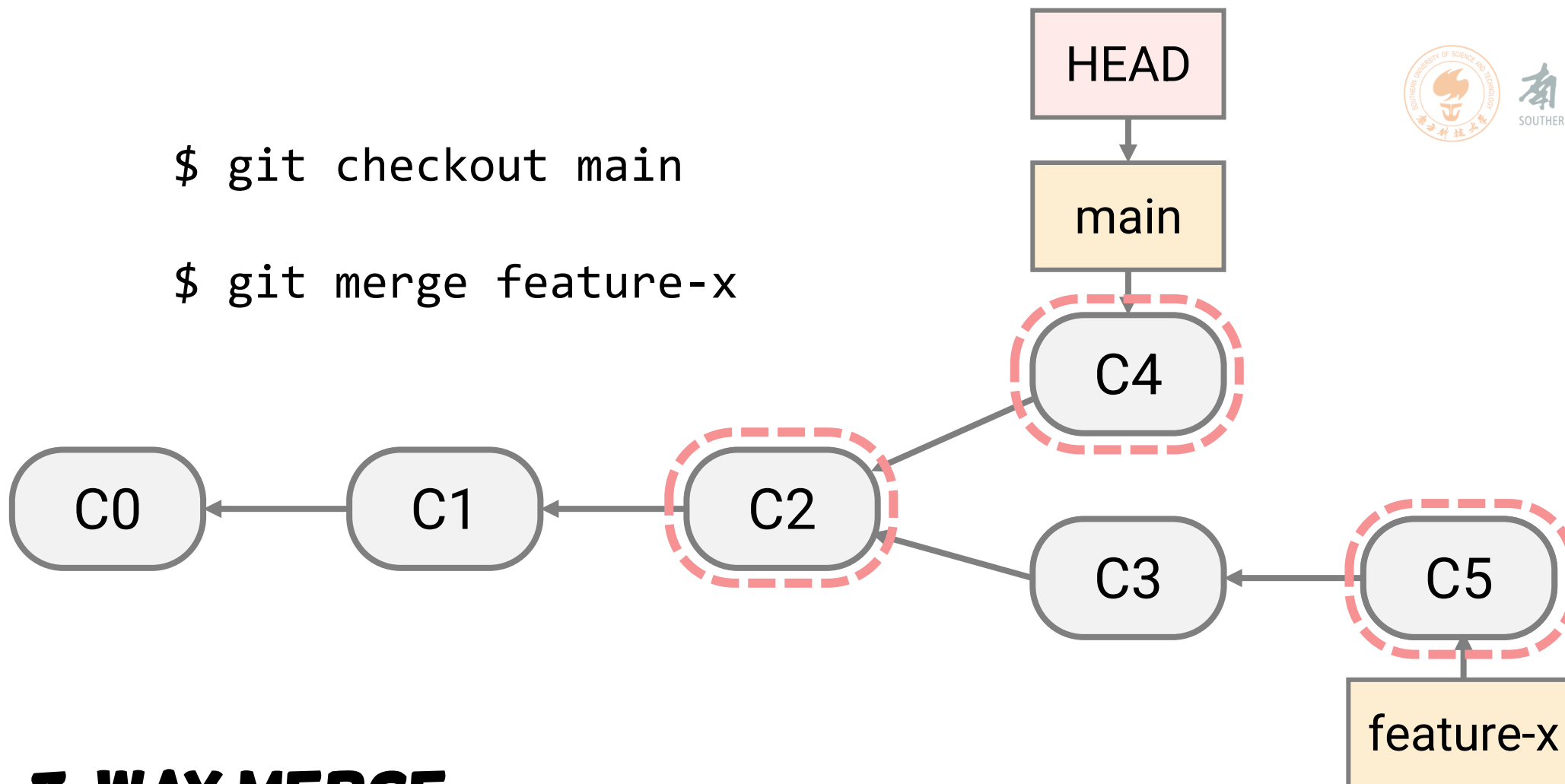


MERGE EXAMPLE



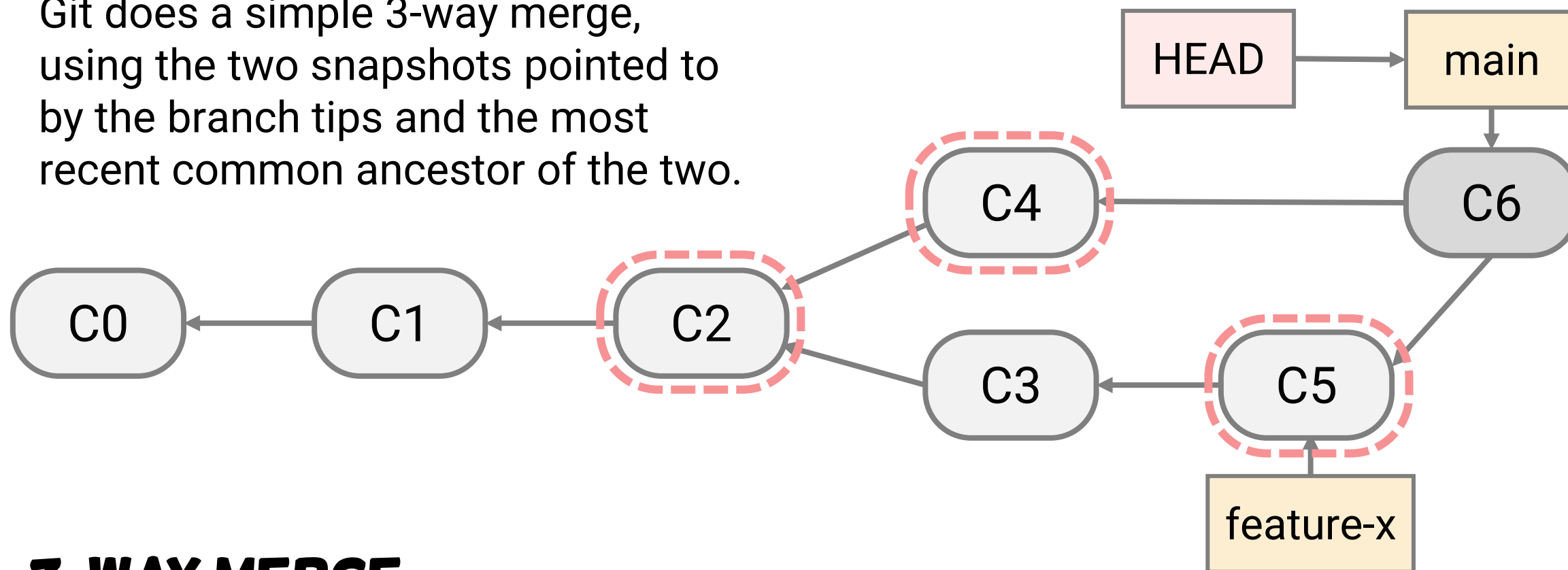
```
$ git checkout main
```

```
$ git merge feature-x
```



3-WAY MERGE

Git does a simple 3-way merge, using the two snapshots pointed to by the branch tips and the most recent common ancestor of the two.



3-WAY MERGE

GIT BRANCHING IN ACTION

<https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging>



MERGE CONFLICTS



Resolved by git automatically



Merge conflict. Resolved by developers manually

MERGE CONFLICTS



Base: layout_BASE_60689.tsx
Left: layout_LOCAL_60689.tsx
Right: layout_REMOTE_60689.tsx
Merge: layout.tsx

Differences from base: 9
Differences from base: 1
Conflicts: 1

./src/components/layout_LOCAL_60689.tsx	./src/components/layout_BASE_60689.tsx	./src/components/layout_REMOTE_60689.tsx
<pre>padding: theme.spacing(3), }, footer: { width: "100vw", background: theme.palette.primary.main, padding: theme.spacing(2), display: "block", [theme.breakpoints.up("sm")]: { marginLeft: drawerWidth, }, }, });</pre>	<pre>padding: theme.spacing(3), }, footer: { width: "100vw", background: theme.palette.primary.main, padding: theme.spacing(2), display: "block", [theme.breakpoints.up("sm")]: { marginLeft: props => props.drawerWidth, }, }, });</pre>	<pre>padding: theme.spacing(3), }, footer: { width: "100vw", background: theme.palette.primary.main, padding: theme.spacing(2), display: "block", [theme.breakpoints.up("sm")]: { marginLeft: props => props.drawerWidth, }, }, });</pre>
<pre>const muiCache = makeMuiCache(); const Layout: React.FC<LayoutProps> = ({ children }) => { const drawerWidth = 400; const { classes } = useStyles({ drawerWidth }); const data = useStaticQuery(graphql`     query SiteTitleQuery {       site {         siteMetadata {           title         }       }     }   `); return (<ThemeProvider theme={theme}> <CacheProvider value={muiCache}></pre>	<pre>const Layout: React.FC = ({ children }) => { const drawerWidth = 400; const classes = useStyles({ drawerWidth }); const data = useStaticQuery(graphql`     query SiteTitleQuery {       site {         siteMetadata {           title         }       }     }   `); return (<ThemeProvider theme={theme}> </* CssBaseline kickstart an elegant, consistent, and simple <CssBaseline /></pre>	<pre>const Layout: React.FC = ({ children }) => { const drawerWidth = 300; const classes = useStyles({ drawerWidth }); const data = useStaticQuery(graphql`     query SiteTitleQuery {       site {         siteMetadata {           title         }       }     }   `); return (<ThemeProvider theme={theme}> </* CssBaseline kickstart an elegant, consistent, and simple <CssBaseline /></pre>

When there are conflicting changes, git cannot do merge automatically and you have to resolve the conflict manually.



RESOLVING CONFLICTS

- Conflict markers (<<<<, ====, >>>>) in files
- Edit files to resolve conflicts manually
- `git add <file>`: mark resolved
- `git commit`: finish the merge

```
<<<<<< HEAD  
code from the main branch  
=====  
code from feature-x branch  
>>>>>> feature-x
```

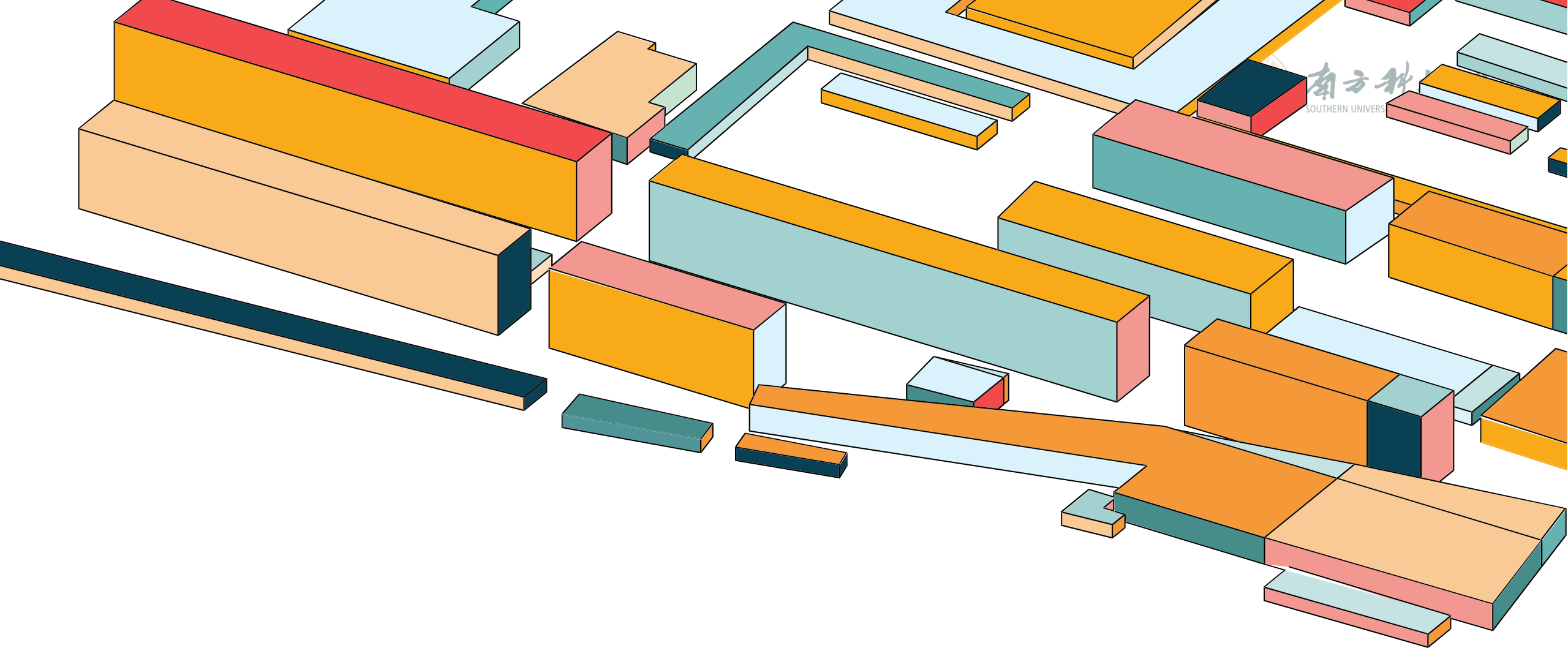
GIT BEST PRACTICES

Commit early, commit often

- It is better to work in small chunks and keep committing your work.
- It helps you from losing work, reverting changes, and helping trace what you did
- It helps you keep your code updated with the latest changes so that you **avoid merge conflicts**.

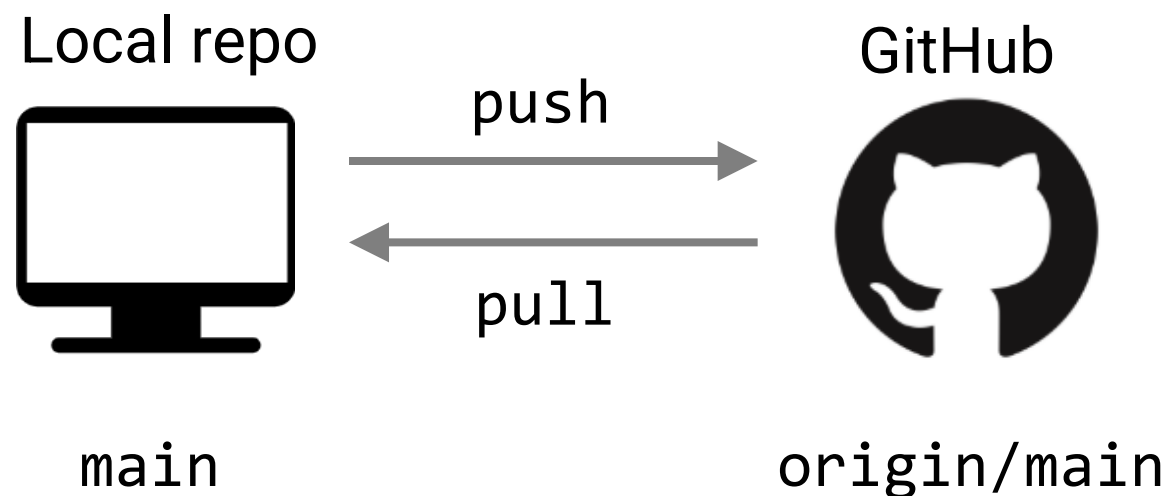
Showing 1 changed file with 8 additions and 12 deletions. Unified Split

```
20 user-manual/add-edit-content/authority-records.rst
@@ -262,12 +262,10 @@ Add a new authority record from the main menu
262 262
263 263 3. Enter data as required. The authority record edit template is based on the
264 264 `ICA's <http://www.ica.org/>` *International Standard Archival Authority
265 - Record for Corporate Bodies, Persons and Families* (`ISAAR
266 - <http://www.ica.org/10203/standards/isaar-cpf- international-standard-
267 - archival-authority-record-for-corporate-bodies-persons- and-families-2nd-
268 - edition.html>`). For more information on the use of particular fields in
269 - the ISAAR authority record edit template, see: :ref:`ISAAR(CPF)
270 - <isaar-template>`.
265 + Record for Corporate Bodies, Persons and Families*
266 + (`ISAAR <https://www.ica.org/en/isaar-cpf-international-standard-archival-authority-record-corporate-bodies-persons-and-
267 + families-2nd>`).
268 + For more information on the use of particular fields in the ISAAR authority
269 + record edit template, see: :ref:`ISAAR(CPF) <isaar-template>`.
271 269 4. You can quit the creation process at any time by clicking the "Cancel"
272 270 button in the :term:`button block`; any data already entered will not be
273 271 saved, and no new record will be created. Note that simply navigating away
@@ -498,12 +496,10 @@ page` of any other authority record via the "Add new" button in the
498 496
499 497 3. Enter data as required. The authority record edit template is based on the
500 498 `ICA's <http://www.ica.org/>` *International Standard Archival Authority
501 - Record for Corporate Bodies, Persons and Families* (`ISAAR
502 - <http://www.ica.org/10203/standards/isaar-cpf- international-standard-
503 - archival-authority-record-for-corporate-bodies-persons- and-families-2nd-
504 - edition.html>`). For more information on the use of particular fields in
505 - the ISAAR authority record edit template, see: :ref:`ISAAR(CPF)
506 - <isaar-template>`.
499 + Record for Corporate Bodies, Persons and Families*
500 + (`ISAAR <https://www.ica.org/en/isaar-cpf-international-standard-archival-authority-record-corporate-bodies-persons-and-
501 + families-2nd>`).
```



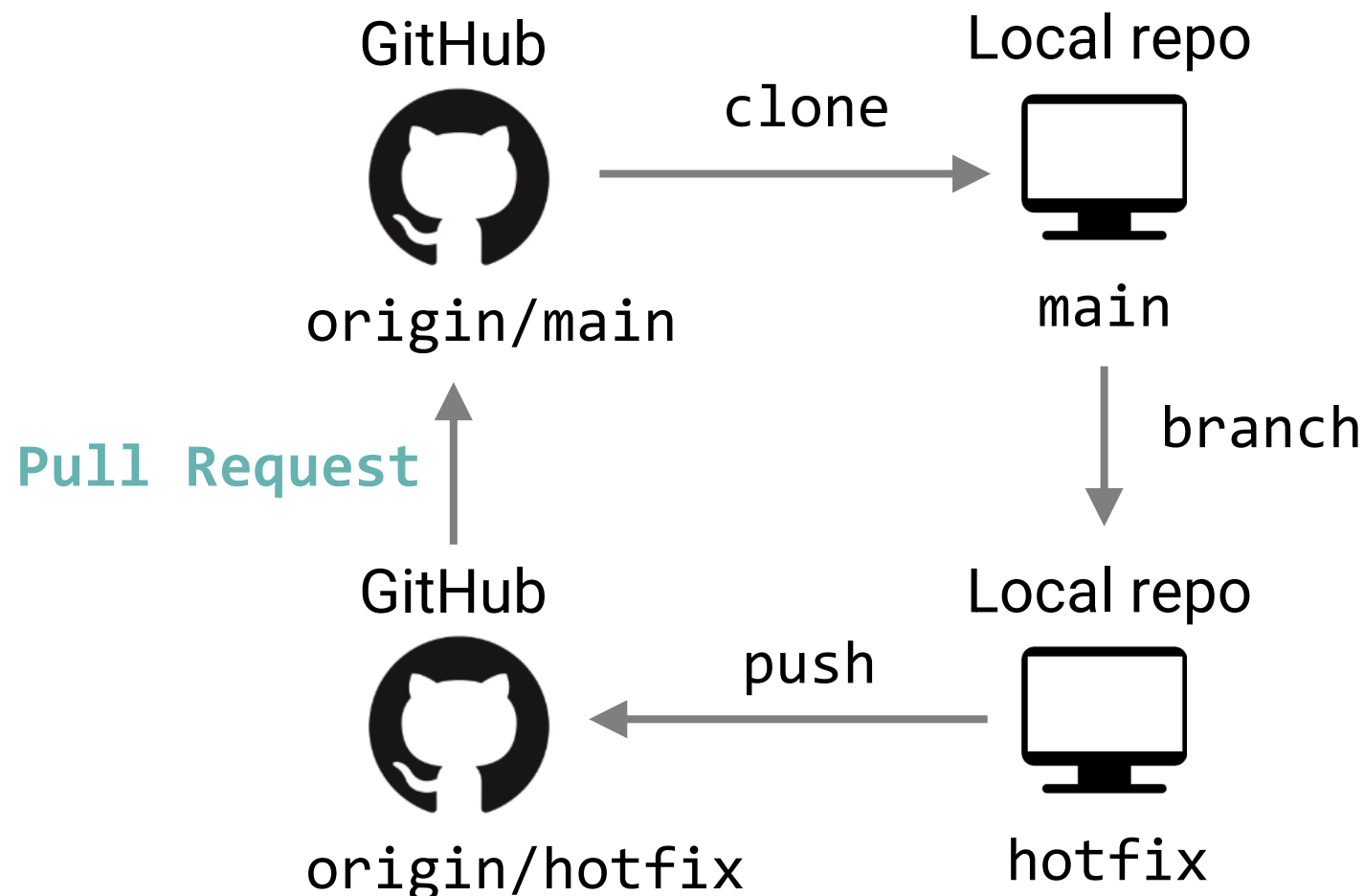
OPEN-SOURCE CONTRIBUTION WORKFLOW

NO-BRANCH WORKFLOW



PR WORKFLOW

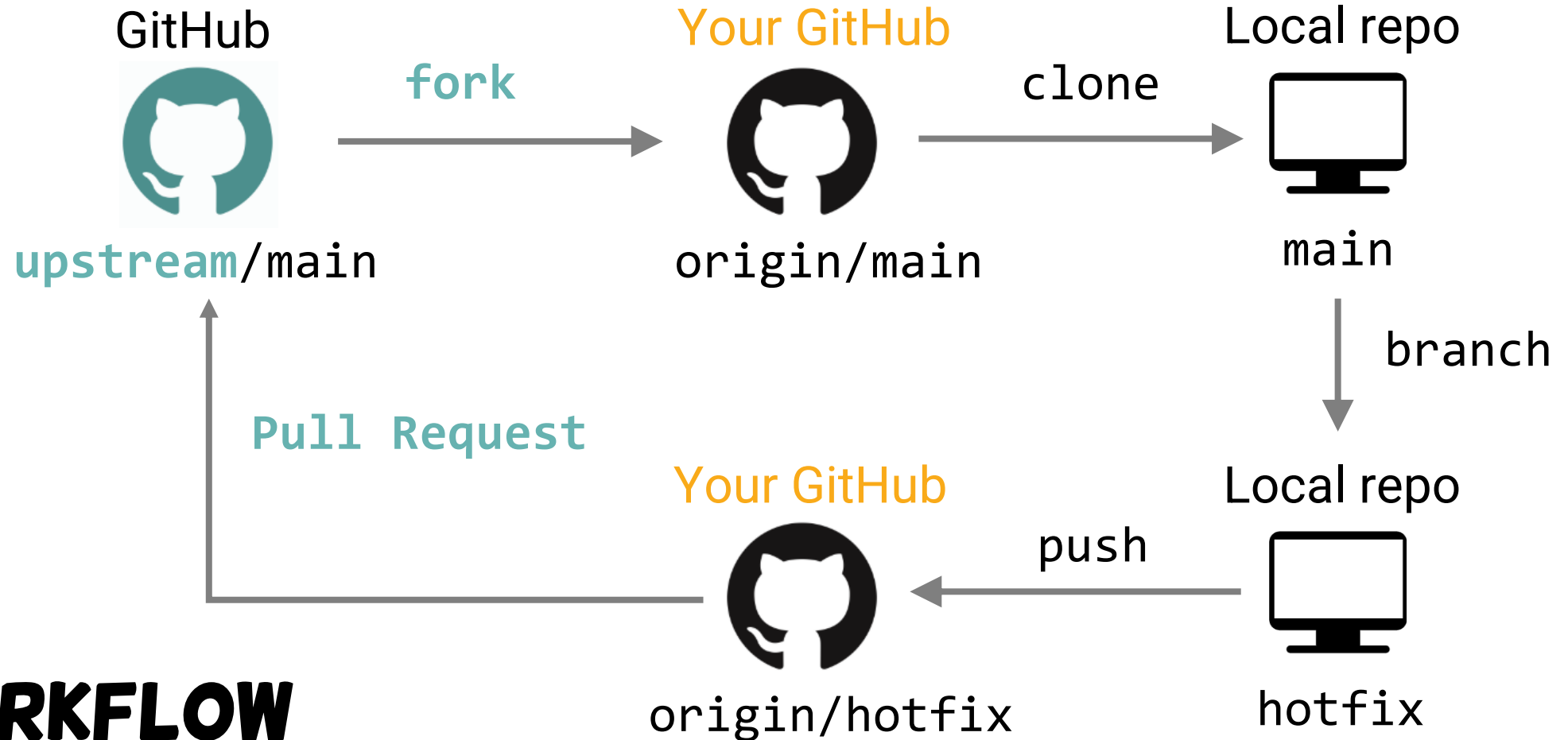
A pull request (PR) is a proposal to merge a set of changes from one branch into another.



A fork will create a completely new copy of the target repository (**upstream** repo).

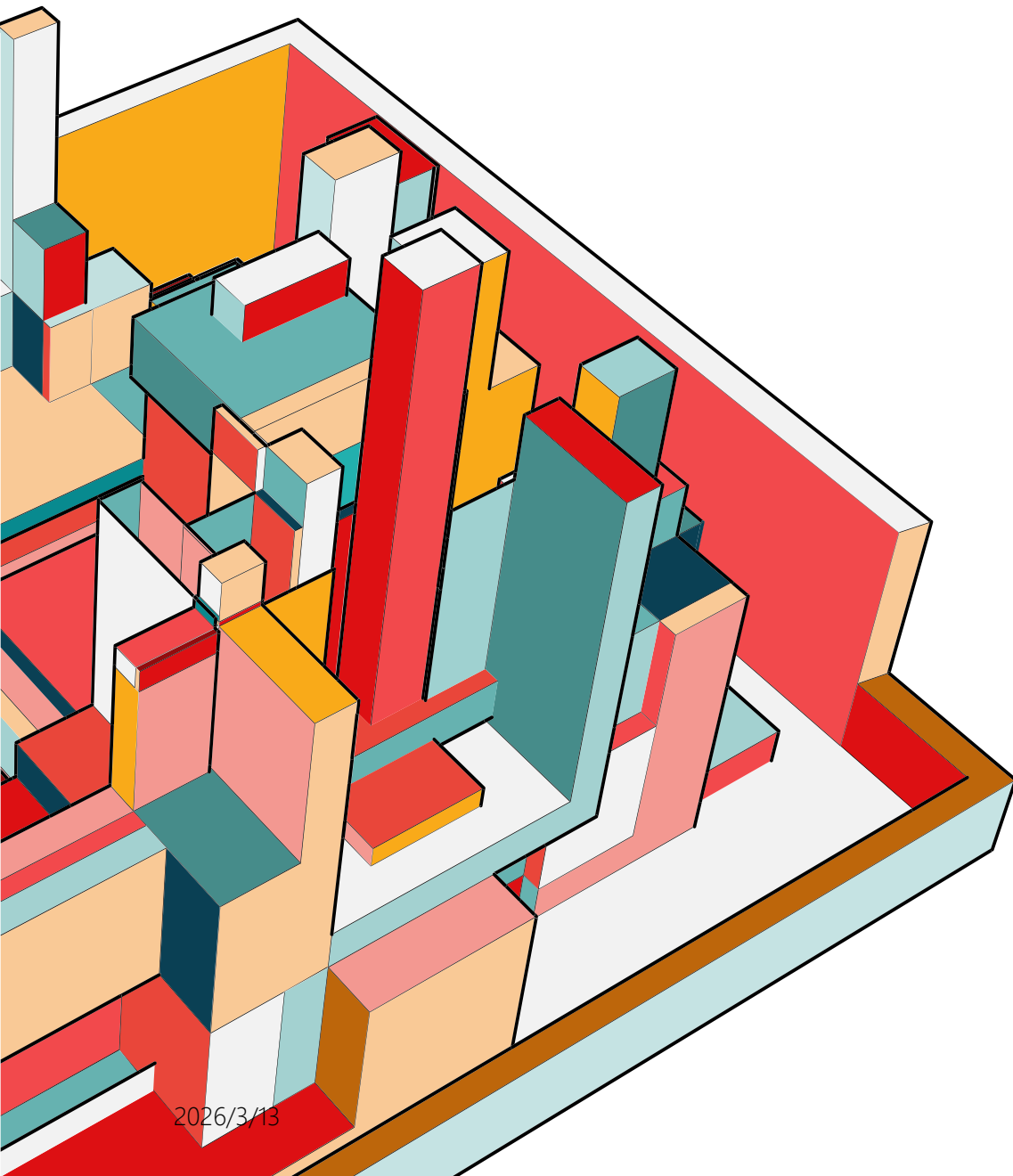


Developers who contributed to the upstream repo cannot contribute to the new fork unless access granted



FORK WORKFLOW

Developers who performs the fork will have complete control over the newly copied codebase.



LET'S FOLLOW GIT BEST PRACTICE IN

- Team project
- Individual assignment: open-source contribution



NEXT

- Software Architecture